COLLABRIX

A Team-First Freelance Collaboration Platform

A Project Report

Submitted by

Jaghatha Uday Prakash
2303031240447

Boddu Venkata Sai Kowshik
2303031241591

Katreddi Hima Nagendra
2303031240520

Immadi Venkata Rama Chaitanya Vardhan
2303031240436

In Partial Fulfilment for the
Award of the Degree of

BACHELOR OF TECHNOLOGY

COMPUTER SCIENCE AND ENGINEERING (Spl. In Artificial Intelligence)

Under the Guidance of

Assistant Prof. Barkha Makwana

PARUL UNIVERSITY, VADODARA

October – 2026

PARUL UNIVERSITY

CERTIFICATE

This is to certify that Project-2, Subject Code 303105423 of the 7th Semester, entitled **"CollabriX: A Team-First Freelance Collaboration Platform"** of Group No. PIET CSE NON ASMP - 8 has been completed by

- JAGHATHA UDAY PRAKASH – 2303031240447
- BODDU VENKATA SAI KOWSHIK – 2303031241591
- KATREDDI HIMA NAGENDRA – 2303031240520
- IMMADI VENKATA RAMA CHAITANYA VARDHAN – 2303031240436

under my guidance, in partial fulfilment of the Bachelor of Technology (B. Tech) in Computer Science and Engineering at Parul University in the Academic Year 2026–2027.

Date of Submission: _____

Project Guide
Assistant Prof. Barkha Makwana

Head of Department,
Dr. Sanjay Agal
Department of Artificial Intelligence and Data Science, PIET
Parul University, Vadodara

Project Coordinator
Dr. Mohammad Arif

External Examiner

Acknowledgements

"Alone we can do so little; together we can do so much."
– Helen Keller

It gives us great pleasure to present this report on **CollabriX**, a platform that is itself about people working better together. Building it taught us as much about collaboration as about software, and we owe that lesson to many people.

First and foremost, we express our sincere gratitude to our project guide, **Assistant Prof. Barkha Makwana**, for her constant encouragement, patient reviews and valuable suggestions at every stage of this work. Her questions about scope, security and evaluation shaped the direction of the project and of this report.

We thank **Dr. Sanjay Agal**, Head of the Department of Artificial Intelligence and Data Science, PIET, and **Dr. Mohammad Arif**, Project Coordinator, for providing the facilities, guidelines and timely reviews that made this project possible.

We are grateful to our parents and families for their unwavering support and belief in us, and to our friends and classmates for their constant encouragement.

Finally, we thank all the faculty and staff of Parul Institute of Engineering and Technology who helped us directly or indirectly during the course of this project.

Jaghatha Uday Prakash
Boddu Venkata Sai Kowshik
Katreddi Hima Nagendra
Immadi Venkata Rama Chaitanya Vardhan
B.Tech. – CSE (AI)
Parul University, Vadodara

Abstract

Freelance marketplaces such as Upwork, Fiverr and Freelancer have made it easy for an individual to sell a skill to a client anywhere in the world. They are, however, designed around a single seller competing against other single sellers through bids. Larger projects need a front-end developer, a back-end developer and a designer who already trust each other, and today such teams are assembled informally, outside the platform, through personal messages and spreadsheets. Reputation is also fragile: ratings are often inflated, and they are frequently disconnected from the actual work that was delivered.

CollabriX is a web-based, team-first freelance collaboration platform built to address these gaps. It provides two role-based workspaces. **Developers** browse open projects, discover peers by skill, region and rating, exchange connection requests that the receiver accepts or rejects, chat in real time, assemble *team batches* from accepted connections with an automatically computed synergy score, answer project invitations and climb a leaderboard computed only from verified work. **Clients** post projects, search for talent with faceted filters, invite developers, assign one who accepted, mark the work complete and rate it. Both roles receive real-time notifications.

The system is a single-page application in **React 18, TypeScript and Vite**, styled with **Tailwind CSS** and **shadcn/ui** components, with **TanStack Query** for server-state caching. The back end is **Supabase**: PostgreSQL with twelve application tables, **Row-Level Security (RLS)** on every table, and triggers and security-definer functions that enforce each workflow rule inside the database. A developer is never assigned merely because an invitation was sent, a rating requires a completed project shared by both parties, and notifications can only be produced by the database itself. Realtime subscriptions push messages, requests and notifications without polling.

The prototype was built in five agile sprints. The final build passes the TypeScript compiler with no errors and ESLint with no errors (down from 14), all 18 automated browser test cases, and a database test suite of 104 behaviour and security checks run against all eleven migrations, including a rehearsal of the upgrade on a copy of the live schema with legacy data. A Lighthouse audit of the production build gives Performance 76, Accessibility 84, Best Practices 100 and SEO 100. Remaining limitations are documented with future work.

Table of Contents

Acknowledgements iii
Abstract iv
Table of Contents v
List of Tables vii
List of Figures ix
List of Algorithms xi
1 Introduction 1
1.1 Drawbacks of the Existing Solution 2
1.2 Scope 2
1.2.1 Key Features and Expected Outcomes 3
1.3 Aim and Objectives 3
1.4 Organisation of the Report 4
2 Literature Survey 5
2.1 Critical Evaluation of Available Literature 5
2.2 Other Platforms Reviewed 7
2.3 Summary of the Literature 8
2.4 Research Gap and Proposed Solution 8
3 Project Flow and Methodology 9
3.1 Software Dependencies 9
3.1.1 Technology Stack 9
3.1.2 Hardware and Software Requirements 10
3.2 Requirement Analysis 10
3.2.1 Intended Users 10
3.2.2 Functional Requirements 11
3.2.3 Non-Functional Requirements 12
3.2.4 User Interface Requirements 13
3.2.5 Assumptions and Constraints 13
3.3 Development Methodology 13
3.3.1 Design Thinking and User-Centric Principles 14
3.3.2 Quality Assurance Strategy 15
3.3.3 Tools for Collaboration 15
3.3.4 Project Timeline (Gantt Chart) 15
3.4 System Architecture 16
3.5 Database Design 17
3.6 UML Design Diagrams 20
3.6.1 Use Case Diagram 20
3.6.2 Sequence Diagram — Registration and Onboarding 20

3.6.3 Sequence Diagram — Connection and Real-time Messaging 21
3.6.4 Navigation Design 22
3.6.5 State Diagram — Connection Requests 22
3.6.6 State Diagram — Project and Invitation Lifecycle 23
3.6.7 Sequence Diagram — From Invitation to Rating 23
3.7 User Interface Design 24
3.8 Communication Profile 24
3.9 Security Design 25
3.9.1 Workflow Rules Enforced by Triggers 27
3.9.2 Trusted Operations and Security-Definer Functions 28
3.9.3 Threat Model Review 29
3.10 Module Implementation 30
3.10.1 Project Structure 30
3.10.2 Authentication and Role Routing 30
3.10.3 Onboarding and Profiles 31
3.10.4 Developer Discovery, Connections and Requests 32
3.10.5 Real-time Messaging 32
3.10.6 Team Formation Engine 33
3.10.7 Projects, Ratings and Leaderboard 34
3.10.8 Project Posting and Management 35
3.10.9 Invitations and Assignment 36
3.10.10 Notification System 36
3.10.11 Settings and Account Management 37
3.10.12 Contact Support 37
3.10.13 Error Handling and User Feedback 38
3.11 Configuration and Deployment 38
4 Results and Analysis 39
4.1 Tools and Technologies 39
4.2 Results 39
4.2.1 Quantitative Results 42
5 Conclusion and Future Works 51
5.1 Conclusion 51
5.1.1 Lessons Learned 52
5.2 Limitations 52
5.3 Future Prospectus 53
5.3.1 Possible Integration with Other Tools 53
Bibliography 54

List of Tables

1.1 Scope of the CollabriX prototype 2
1.2 Key features of CollabriX and their expected outcomes 3
2.1 Comparison of existing freelance platforms with CollabriX 7
2.2 Summary of reviewed literature 8
3.1 Technology stack and justification 9
3.2 Hardware requirements 10
3.3 Software requirements 10
3.4 Illustrative user personas 10
3.5 Functional requirements 11
3.6 Non-functional requirements 12
3.7 Sprint plan and deliverables (dates from the Git history) 14
3.8 Collaboration and development tools 15
3.9 Architectural layers and responsibilities 16
3.10 Description of database tables 17
3.11 Data dictionary of the profiles table 18
3.12 Data dictionary of the notifications table 19
3.13 Communication channels used by CollabriX 25
3.14 Row-Level Security policies implemented 25
3.15 Database functions exposed to or used by the application 28
3.16 Threats considered and their mitigations 29
3.17 Source code organisation 30
3.18 Relationship states and the actions offered 32
3.19 Worked example of the leaderboard formula 35
3.20 Validation rules of the Post Project form 36
3.21 Notification types, their triggers and the preference that controls them 37
3.22 Mapping of technical errors to user-facing messages 38
4.1 Tools and technologies used 39
4.2 Comparative analysis of conventional freelance platforms vs. CollabriX 39
4.3 Static analysis results 43

4.4 Automated UI test cases and results 43
4.5 Manual test cases for authenticated features 45
4.6 RLS validation with an anonymous client 46
4.7 Lighthouse metrics and bundle sizes 47
4.8 Database test suite: areas and results 48
4.9 Upgrade rehearsal with legacy data 49
4.10 Defects, root causes and fixes 50
5.1 Project goals versus accomplishments 51
5.2 Known limitations of the current prototype 52

List of Figures

1.1 Overview of the CollabriX collaboration ecosystem 1

3.1 Iterative sprint cycle followed in the project 14

3.2 Gantt chart of the project timeline 15

3.3 Layered system architecture of CollabriX 16

3.4 Entity–relationship diagram 17

3.5 Entity–relationship diagram of the collaboration extension 19

3.6 Use case diagram 20

3.7 Sequence diagram for registration and onboarding 21

3.8 Sequence diagram for connection request and messaging 21

3.9 Role-based navigation map 22

3.10 State diagram of a connection request 23

3.11 State diagrams of the project and invitation lifecycles 23

3.12 Sequence diagram from invitation to two-way rating 24

3.13 Flowchart of authentication and role-based routing 31

3.14 Flowchart of the synergy score algorithm 34

3.15 Flowchart of rating submission and validation 34

3.16 Notification pipeline from database event to the user interface 37

4.1 Splash screen shown while the session is restored 40

4.2 Login screen with e-mail/password and Google sign-in 40

4.3 Sign-up screen with Developer / Client role selection 41

4.4 Developer workspace: (a) profile with ratings and badges, (b) real-time chat with a connection 41

4.5 (a) Team Formation Engine with synergy score and skill gaps, (b) client dashboard 42

4.6 (a) Notifications page, (b) project details with invitations and the Assign action 42

4.7 ESLint findings before and after the fixes 43

4.8 Sign-up screen rendered at mobile width (TC-09).....45

4.9 Lighthouse category scores before and after the extension 47

4.10 Database test suite results by area.....49

List of Algorithms

1 Synergy Score Calculation for a Team Batch 33
2 Leaderboard Ranking (database function get_leaderboard) 35
3 Invitation-gated assignment of a developer 36

Chapter 1

Introduction

The way software is built has changed. A modern product is rarely written by one person; it is assembled by small cross-functional teams in which a front-end engineer, a back-end engineer, a designer and someone responsible for deployment work together for a few weeks or months. At the same time, an increasing share of this work is performed by independent professionals who are hired online. The freelance or *gig* economy has therefore become an important channel through which companies, especially start-ups and small businesses, access technical talent.

This project, **CollabriX**, is a web platform that connects clients who need software built with developers who build it, and, crucially, lets developers find each other and form teams on the platform itself. The name combines *collaboration* with *X*, indicating that collaboration is the core of the product rather than an afterthought.

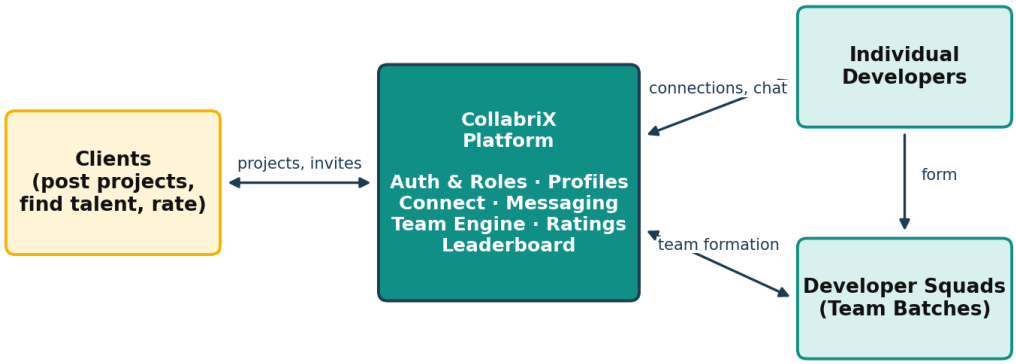


Figure 1.1: Overview of the CollabriX collaboration ecosystem

Online labour markets are two-sided platforms: one side (clients) posts work and the other side (freelancers) offers skills. The platform earns trust by verifying identities, providing communication tools, and maintaining reputation systems such as star ratings and job success scores. Research on online labour markets shows that reputation is the primary signal clients use when choosing whom to hire, and that new entrants without a track record find it hard to win their first contracts [1], [2].

Technically, platforms in this domain are data-intensive, multi-tenant web applications. Every user must only see and modify the data that belongs to them; conversations must be delivered instantly; profile, project and rating data must be consistent. These requirements map naturally onto a relational database with strong access control and a realtime messaging layer, which is the architecture CollabriX adopts.

Student and early-career developers commonly face the same situation: a client asks for a complete application, but no single person can deliver every part alone. Finding a reliable partner usually means asking around in college or social groups, and once a team is formed there is no shared place to talk to the client, track the project or build a joint reputation. Research on *flash teams* and *flash organisations* [6], [7] shows that experts recruited online can deliver complex work when the platform gives them structure. CollabriX applies that idea to everyday freelance development.

The project is relevant to the Indian context in particular, where a large population of young developers is entering the global freelance market and where team-based delivery can help them compete for larger, better-paid contracts.

1.1 Drawbacks of the Existing Solution

1. **Individual-only model:** mainstream marketplaces are optimised for one freelancer bidding on one job. There is no first-class way for developers to discover complementary peers and apply to a project as a team.
2. **Bidding fatigue:** clients receive many generic proposals and must filter them manually, while freelancers spend unpaid time writing bids.
3. **Weak link between ratings and work:** reviews tend to be inflated over time [8] and are not always tied in a verifiable way to a completed engagement, which reduces their value as a quality signal.
4. **Fragmented communication:** conversations, team coordination and project status often move to external tools such as e-mail or chat apps, so the platform loses context.
5. **Cold-start problem:** newcomers without reviews are almost invisible in search results, regardless of their actual skills [2].

1.2 Scope

The scope of the project is a working web prototype of the platform, including both user roles, the developer networking features and the supporting database and security layer. Table 1.1 summarises what is included and what is deliberately excluded.

Table 1.1: Scope of the CollabriX prototype

In scope	Out of scope (for this phase)
Role-based sign-up and login (e-mail/password and Google OAuth)	Payments, escrow and invoicing
Developer and client onboarding wizards and profile editing	Native Android / iOS applications
Developer search with type, skill and region filters	Automated résumé or LinkedIn parsing
Connection requests and real-time one-to-one messaging	Video calls and file sharing in chat
Team batch formation with synergy scoring	Activity tracking / screenshot monitoring
Completion-gated project and user ratings, leaderboard, badges	Dispute resolution and moderation tools
Row-Level Security for every application table	Multi-language (i18n) support

In scope	Out of scope (for this phase)
Project posting, invitations, assignment and completion	Proposals, bidding and contracts
Real-time notifications, settings and support tickets	Staff console for support and moderation

1.2.1 Key Features and Expected Outcomes

Table 1.2: Key features of CollabriX and their expected outcomes

Feature	Description	Expected outcome
Role-based workspaces	Separate bottom-navigation and screens for developers (5 tabs) and clients (4 tabs)	Each user sees only relevant tools
Guided onboarding	Three-step developer and company setup wizards with live preview	Complete profiles from day one
Developer discovery	Search by name, skill, type; continent → country region filter; rating badge	Faster peer and talent discovery
Connections & messaging	Request/accept workflow, unread badges, realtime chat threads	In-platform communication
Team Formation Engine	Batches built from accepted connections with role assignment and synergy %	Balanced, complementary teams
Ratings & leaderboard	1–5 star ratings tied to completed projects; ranking and badges	Trustworthy reputation signal
Security by design	RLS policies, JWT sessions, per-user storage folders	Zero-trust data access
Project lifecycle	Post, edit, invite, assign a developer who accepted, mark complete	Ratings flow from real, finished work
Notifications	Generated by database triggers, delivered in real time, per-type preferences	No missed requests or invitations
Settings and support	Profile, privacy, account security and ticketed help	Self-service account management

1.3 Aim and Objectives

1. To design and implement a responsive single-page web application with separate, role-specific workspaces for developers and clients.
2. To enable developers to discover peers by skill, developer type and region, and to connect with them through a request–accept workflow.
3. To provide real-time messaging between connected users without polling.
4. To let developers assemble team batches from their verified connections and see an objective skill-coverage score for the team.
5. To build a reputation system in which ratings can only be submitted for completed projects by the people who took part in them.

6. To provide a complete project lifecycle (posting, invitation, assignment and completion) whose rules are enforced by the database, so that ratings come only from real engagements.
7. To keep every user informed through real-time notifications that respect individual preferences.
8. To enforce all authorisation rules at the database level using PostgreSQL Row-Level Security, so that the browser never has to be trusted.
9. To evaluate the system through functional, security and performance testing and to document its limitations.

1.4 Organisation of the Report

Chapter 2 evaluates related research and existing platforms. Chapter 3 covers the project flow and methodology, from requirements and design to the implementation of each module, including the collaboration extension (notifications, invitations and the project lifecycle) added in the final sprint. Chapter 4 presents the tools used, screenshots and the results of static, browser, database and performance tests. Chapter 5 concludes the report and outlines future work.

Chapter 2

Literature Survey

This chapter reviews research on online labour markets, reputation systems and team formation, followed by an analysis of popular freelance platforms. The goal is to identify what is already known, what existing systems do well, and where a team-first platform can contribute.

2.1 Critical Evaluation of Available Literature

2.1.1 Paper 1: "Online Labor Markets" by J. J. Horton [1]

Horton describes online labour markets as markets in which workers and buyers are matched over the internet and work is delivered electronically. He discusses their economic properties: low search costs, global participation, and the strong role that the platform plays as a market designer. A key observation is that the platform's rules — how jobs are posted, how workers are ranked and how feedback is collected — directly shape outcomes for both sides.

Relevance: CollabriX treats these design choices (who may message whom, who may rate whom) as explicit, enforced rules rather than conventions.

2.1.2 Paper 2: "The Future of Crowd Work" by A. Kittur, J. V. Nickerson, M. Bernstein et al. [2]

Kittur and colleagues analyse paid crowd work and propose a research agenda for making it a career that people would want for their children. They identify the need for better task decomposition, worker collaboration, reputation and credentialing, and support for complex, interdependent work rather than isolated micro-tasks.

Relevance: the paper motivates CollabriX's focus on collaboration between workers and on reputation that reflects real outcomes.

2.1.3 Paper 3: "Reputation Systems" by P. Resnick, K. Kuwabara, R. Zeckhauser and E. Friedman [3]

This influential article explains how reputation systems build trust between strangers by collecting, distributing and aggregating feedback about past behaviour. The authors note three challenges: people may not leave feedback, negative feedback is hard to elicit, and it is hard to ensure that feedback is honest. They argue that the system must make feedback both available and credible.

Relevance: CollabriX addresses credibility by making it impossible, at the database level, to rate a user or project unless a completed engagement exists.

2.1.4 Paper 4: "A Survey of Trust and Reputation Systems for Online Service Provision" by A. Jøsang, R. Ismail and C. Boyd [4]

Jøsang, Ismail and Boyd survey trust and reputation models used in online services, from simple summation and averaging (as on eBay) to Bayesian and flow-based models. They also

catalogue attacks such as ballot stuffing, bad-mouthing and identity changes, and discuss how system design can mitigate them.

Relevance: averaging is used in the prototype for simplicity; unique constraints prevent the same rater from rating the same project twice, which mitigates ballot stuffing.

2.1.5 Paper 5: "Finding a Team of Experts in Social Networks" by T. Lappas, K. Liu and E. Terzi [5]

Lappas, Liu and Terzi formalise the *team formation problem*: given a task requiring a set of skills and a social network of experts, find a team that covers all required skills while minimising the communication cost between members. They prove the problem is NP-hard and propose approximation algorithms based on the diameter and minimum spanning tree of the team in the network.

Relevance: CollabriX restricts team members to a developer's accepted connections (existing social ties) and measures skill coverage against domain requirements — a practical, explainable version of this formulation.

2.1.6 Paper 6: "Online Team Formation in Social Networks" by A. Anagnostopoulos, L. Becchetti, C. Castillo, A. Gionis and S. Leonardi [11]

This work extends team formation to an online setting in which tasks arrive over time and the system must assign teams while balancing the workload of experts. The authors show that good solutions must consider both skill coverage and fairness of load across people.

Relevance: the profiles table already stores a `current_projects` count, which can be used in future to balance workload during team suggestions.

2.1.7 Paper 7: "Expert Crowdsourcing with Flash Teams" by D. Retelny, S. Robaszkiewicz, A. To et al. [6]

Flash teams are computationally guided teams of crowd experts assembled on demand. Using a framework called Foundry, the authors modelled workflows as linked modular tasks and recruited experts from an online marketplace; flash teams completed complex tasks such as app design considerably faster than traditional self-managed teams.

Relevance: demonstrates that structured, on-platform team assembly can outperform ad hoc coordination, which is the core hypothesis of CollabriX.

2.1.8 Paper 8: "Flash Organizations: Crowdsourcing Complex Work by Structuring Crowds as Organizations" by M. A. Valentine, D. Retelny, A. To et al. [7]

Flash organisations extend flash teams with organisational structures — roles, hierarchies and the ability to change the structure while work is in progress. Crowd workers in these organisations built complex products such as a mobile application. The work highlights the importance of explicit roles.

Relevance: CollabriX lets the batch creator assign explicit roles (Team Lead, Frontend Engineer, Backend Engineer, etc.) to each member.

2.1.9 Paper 9: "Reputation Inflation" by A. Filippas, J. J. Horton and J. Golden [8]

Using data from a large online labour market, the authors show that average feedback scores rose substantially over time while underlying quality did not, because giving negative

feedback is costly to the rater. As a result, ratings lose their ability to distinguish good from average workers.

Relevance: motivates multi-dimensional and completion-tied ratings, and future work on private (double-blind) feedback in CollabriX.

2.1.10 Paper 10: "Reputation Transferability in Online Labor Markets" by M. Kokkodis and P. G. Ipeirotis [9]

Kokkodis and Ipeirotis study how a worker's reputation in one category of work predicts performance in another, and propose models that transfer reputation across task categories. They find that naive aggregation can mislead employers when workers switch domains.

Relevance: supports showing skill-specific information (skills, developer type) alongside a single average rating.

2.1.11 Paper 11: "The Digitization of Word of Mouth" by C. Dellarocas [12]

Dellarocas reviews online feedback mechanisms and contrasts them with traditional word of mouth. Online systems operate at a very large scale, and, crucially, their designers control what is collected, who may contribute and how it is aggregated and displayed. The paper discusses strategic manipulation, the cost of leaving feedback and the effect of anonymous or easily changed identities, and argues that these design parameters determine whether a feedback system is informative.

Relevance: CollabriX uses this control deliberately. Only the two parties of a completed project may rate each other, the direction of every rating is recorded, ratings cannot be edited afterwards, and client and developer ratings are kept separate on profiles.

2.1.12 Paper 12: "Good Gig, Bad Gig" by A. J. Wood, M. Graham, V. Lehdonvirta and I. Hjorth [10]

Based on interviews and a survey of remote gig workers in South-East Asia and Sub-Saharan Africa, the authors show that platform work offers flexibility and autonomy but also brings algorithmic control through ratings and rankings, together with overwork, social isolation and insecurity. Workers experience opaque reputation scores as a powerful form of control.

Relevance: the leaderboard formula is public and explained on screen ("How scores work"), developers can hide themselves from the leaderboard, and notification preferences let users limit interruptions. Team batches also address isolation by making collaboration a first-class feature.

2.2 Other Platforms Reviewed

Four widely used platforms were studied from the perspective of both a client and a developer. Table 2.1 summarises the comparison against the features that matter for team-based delivery.

Table 2.1: Comparison of existing freelance platforms with CollabriX

Aspect	Upwork	Fiverr	Freelancer	Toptal	CollabriX
Hiring model	Job post + proposals	Fixed gigs / packages	Bidding + contests	Curated matching	Profiles + teams

Aspect	Upwork	Fiverr	Freelancer	Toptal	CollabriX
Peer discovery between freelancers	Limited	No	Limited	No	Yes (search + connect)
Form a team on-platform	Agencies only	No	No	Matched by staff	Yes (team batches)
Skill-coverage analysis	No	No	No	Manual	Synergy score
In-platform chat	Yes	Yes	Yes	Yes	Yes (realtime)
Ratings tied to completed work	Yes	Yes	Yes	N/A	Enforced by RLS
Open leaderboard	No	Seller levels	Rankings	No	Yes + badges

2.3 Summary of the Literature

Table 2.2: Summary of reviewed literature

Ref.	Focus	Key finding	Gap addressed by CollabriX
[1]	Online labour markets	Platform rules shape market outcomes	Rules enforced in the database
[2]	Crowd work agenda	Need collaboration & reputation	Team-first design
[3], [4]	Reputation systems	Feedback must be credible; attacks exist	Completion-gated, unique ratings
[5], [11]	Team formation	Cover skills, low communication cost	Teams from existing connections + coverage score
[6], [7]	Flash teams / organisations	Structured expert teams deliver complex work	Explicit roles in batches
[8]	Reputation inflation	Average scores drift upward	Basis for future double-blind ratings
[9]	Reputation transfer	Category matters	Skills shown with ratings
[10]	Gig work and control	Ratings act as algorithmic control	Transparent formula, opt-out
[12]	Online feedback design	Designers control what is collected	Participant-only, directional ratings

2.4 Research Gap and Proposed Solution

The literature establishes that (i) structured teams of online experts can deliver complex work, (ii) team formation should consider both skills and social ties, and (iii) reputation must be credible to be useful. Existing commercial platforms, however, remain largely individual-centric and do not let freelancers discover peers, form teams and evaluate their skill coverage within the platform. CollabriX fills this gap with a connection-based team formation engine, completion-tied ratings enforced in the database, and integrated real-time communication.

Chapter 3

Project Flow and Methodology

This project aims to design and implement a team-first freelance collaboration platform. To accomplish this goal, we followed a systematic approach consisting of requirement analysis, design, iterative development, testing and evaluation. This chapter describes the software dependencies, the requirements, the development methodology, the architecture and database design, the communication and security model, and the implementation of each module.

3.1 Software Dependencies

CollabriX is a pure software project: it needs no special hardware beyond a development computer, and end users only need a modern web browser. The software stack was chosen to minimise the amount of custom server code while keeping strong typing and security.

3.1.1 Technology Stack

Table 3.1: Technology stack and justification

Layer	Technology	Reason for selection
Language	TypeScript 5.8	Static typing catches errors early; generated database types give end-to-end type safety
UI library	React 18.3	Component model, huge ecosystem, team familiarity
Build tool	Vite 5.4 + SWC	Instant dev server start and fast production builds (≈ 2 s)
Styling	Tailwind CSS 3.4 + shadcn/ui	Design tokens, dark mode, accessible Radix primitives
Server state	TanStack Query 5	Caching, background refetch, mutation invalidation
Routing	React Router 6	Declarative routes and redirects
Validation	Zod 3	Schema-based validation of forms
Back-end	Supabase (PostgreSQL)	Auth, REST, realtime and storage without writing a server; RLS for security
Icons / charts	Lucide, Recharts	Consistent icon set; charting for dashboards
Database testing	PostgreSQL 16 + Python (psycopg)	Runs every migration and 104 security checks locally, without touching the live project
Browser testing	Puppeteer + headless Chrome	Repeatable route, sign-in and layout checks

Alternatives considered were the MERN stack (MongoDB, Express, React, Node) and Firebase. MERN would have required writing and hosting a custom API and authorisation layer; Firebase uses a document database whose security rules are less expressive for relational data such as connections and ratings. Supabase offered a relational model, SQL-based security policies and realtime in one managed service.

3.1.2 Hardware and Software Requirements

Table 3.2: Hardware requirements

Component	Development	End user
Processor	Intel Core i5 / Apple M1 or better	Any modern dual-core CPU or smartphone
Memory	8 GB RAM (16 GB recommended)	2 GB RAM
Storage	2 GB free (Node modules, tooling)	No installation required
Network	Broadband internet	Internet connection (3G or better)

Table 3.3: Software requirements

Category	Software / version
Operating system	macOS, Windows 10+ or Linux (development); any OS with a modern browser (users)
Runtime & package manager	Node.js 18+ and npm 9+
Front-end	React 18.3, TypeScript 5.8, Vite 5.4, Tailwind CSS 3.4, shadcn/ui (Radix UI), TanStack Query 5, React Router 6, Zod 3
Back-end	Supabase (PostgreSQL, Auth, Realtime, Storage), supabase-js 2.88
Browser	Chrome, Edge, Firefox or Safari (latest two versions)
Tools	VS Code, Git and GitHub, Lovable, Supabase Studio, ESLint, Chrome DevTools, Lighthouse, PostgreSQL 16 (local test database), Python 3 + psycopg, Puppeteer

3.2 Requirement Analysis

The purpose of this specification is to describe the functional and non-functional requirements of the CollabriX web application in sufficient detail for design, implementation and testing. It is intended for the project team, the project guide and the evaluators.

3.2.1 Intended Users

CollabriX has two primary user classes and one supporting actor:

- **Developer** — an individual freelance developer, designer or engineer who wants to find projects, connect with peers, form teams and build a reputation.
- **Client** — a company representative, start-up founder or individual who needs software built and wants to find and communicate with talent.
- **Supabase platform** — the managed backend that authenticates users, stores data, enforces security policies and pushes realtime events.

The illustrative personas in the following table were used to keep design decisions focused on real needs.

Table 3.4: Illustrative user personas

Persona	Goals	Pain points today
Riya, final-year student developer	Win first paid projects; work with peers	No reviews yet; can't find a back-end partner
Arjun, freelance full-stack engineer	Take larger contracts with a reliable team	Coordinates teams over personal chat apps
Meera, start-up founder (client)	Hire a small team for an MVP quickly	Too many generic proposals; hard to judge
Rahul, SME IT manager (client)	Find specialists by region and skill	Little trust in inflated ratings

The product scope is a responsive web application usable on desktop and mobile browsers. It covers account management, profiles, discovery, connections, messaging, team formation, ratings and leaderboard, as listed in Table 1.1.

3.2.2 Functional Requirements

Table 3.5: Functional requirements

ID	Requirement	Priority
FR-1	The system shall allow a user to register with full name, e-mail, password and a role (developer or client).	High
FR-2	The system shall allow login with e-mail/password and with Google OAuth.	High
FR-3	The system shall automatically create a profile and role record when a user registers.	High
FR-4	The system shall guide new developers and clients through a three-step onboarding wizard.	Medium
FR-5	The system shall route a logged-in user to the developer or client workspace based on the stored role.	High
FR-6	Developers shall be able to edit profile details, avatar, skills, region and a cover image.	Medium
FR-7	Users shall be able to search developers by name, skill and type, and filter by continent and country.	High
FR-8	A developer shall be able to send a connection request; the receiver shall accept or reject it.	High
FR-9	Connected users shall exchange messages delivered in real time.	High
FR-10	The system shall show unread request and message counts as badges.	Medium
FR-11	Developers shall create team batches from accepted connections, assign roles and view a synergy score.	High
FR-12	Developers shall browse open projects and view projects assigned to them.	Medium
FR-13	Clients shall view a dashboard of their projects, budget totals and hired developers.	Medium
FR-14	Participants of a completed project shall rate the project and the other party (1–5 stars, optional review).	High
FR-15	The system shall rank developers on a leaderboard and award badges based on ratings and activity.	Low

ID	Requirement	Priority
FR-16	Users shall be able to sign out from any screen.	High
FR-17	Clients shall post a project with title, description, category, required skills, budget, deadline and complexity; invalid input shall be rejected before saving.	High
FR-18	Project owners shall edit, cancel or delete their own projects, but only while the project is open.	High
FR-19	Clients shall invite a developer to an open project; the developer shall accept or decline the invitation.	High
FR-20	A client shall assign only a developer who accepted the invitation; sending an invitation shall never assign anyone.	High
FR-21	The client shall mark an in-progress project complete, which unlocks ratings in both directions.	High
FR-22	The system shall notify users of requests, responses, batch membership, invitations, assignment, completion and ratings in real time, with read/unread state.	High
FR-23	Users shall choose which notification types they receive and whether they appear on the leaderboard.	Medium
FR-24	Users shall change their password and e-mail address and sign out of all devices.	Medium
FR-25	Users shall open support tickets and follow their status.	Medium
FR-26	Users shall open a developer's public profile from search, connections, requests and the leaderboard.	Medium

3.2.3 Non-Functional Requirements

Table 3.6: Non-functional requirements

ID	Category	Requirement
NFR-1	Security	Every application table shall have Row-Level Security enabled; unauthenticated clients shall not read private data.
NFR-2	Security	Sessions shall use signed JWTs; secrets other than the public anon/ publishable key shall never reach the browser.
NFR-3	Performance	The login page should reach Lighthouse Performance ≥ 75 and Best Practices ≥ 90 on the production build.
NFR-4	Real-time	Messages and requests shall be pushed over WebSockets; no periodic polling.
NFR-5	Usability	The interface shall be mobile-first, usable at 375 px width without horizontal scrolling.
NFR-6	Accessibility	Text and controls should meet WCAG 2.1 AA contrast where possible; forms have labels.
NFR-7	Maintainability	Code shall be written in TypeScript and pass the TypeScript compiler without errors.
NFR-8	Portability	The front end shall be a static build deployable to any CDN; configuration via environment variables.
NFR-9	Reliability	Network failures shall be reported to the user with clear messages instead of silent failures.

ID	Category	Requirement
NFR-10	Honesty	The interface shall never report success for an operation that failed; errors shall be translated into clear messages.
NFR-11	Integrity	Workflow rules (state transitions and who may act) shall be enforced by database triggers and functions, not only by the user interface.

3.2.4 User Interface Requirements

- A consistent teal colour theme with light and dark modes, Inter font and rounded card components.
- A fixed bottom navigation bar with icons and labels; the active tab is highlighted.
- A sticky header with logo, theme toggle, notification bell with unread count, and account menu.
- Feedback for every action through toast notifications (success and error).
- Empty states with an icon and explanatory text whenever a list has no data.

3.2.5 Assumptions and Constraints

- Users have a valid e-mail address; Google login requires the Google provider to be enabled in Supabase.
- The free tier of Supabase limits database size and realtime connections, which is sufficient for a prototype.
- Payments are out of scope; budgets are informational only.
- Database migrations are applied by a project administrator through the Supabase SQL editor; the browser never holds a service-role key.

3.3 Development Methodology

The project followed an **Agile, iterative approach** inspired by Scrum [24]. After an initial phase of requirement study, literature survey and design, development proceeded in four one-week sprints. Each sprint started with selecting items from a prioritised backlog, continued with design and implementation, and ended with a demo, testing and a commit to the shared Git repository. Agile was preferred over the Waterfall model because requirements — especially around onboarding and team formation — evolved as each sprint build was reviewed.

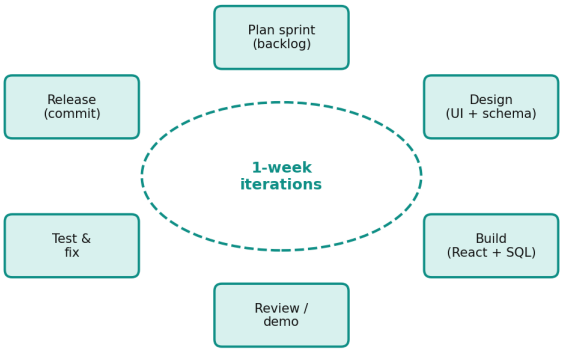


Figure 3.1: Iterative sprint cycle followed in the project

Table 3.7: Sprint plan and deliverables (dates from the Git history)

Sprint	Period	Main deliverables
Sprint 1	16 – 23 Aug 2026	Supabase configuration and fallbacks, .env setup, authentication fixes, role selection
Sprint 2	24 – 31 Aug 2026	Connected messaging and ratings, role-specific profile titles and talent filters, role in sign-up metadata
Sprint 3	1 – 5 Sep 2026	Login page redesign, Google sign-in fix, developer and client onboarding, removal of mock data, cover images
Sprint 4	4 – 17 Sep 2026	Team batch formation engine, CollabriX vs Upwork comparison, configuration restore
Stabilisation	17 – 22 Sep 2026	Bug fixes and build verification
Sprint 5	6 – 12 Oct 2026	Collaboration core migration (notifications, invitations, project lifecycle, settings, support), rework of 20+ screens, database test suite, report update

3.3.1 Design Thinking and User-Centric Principles

We applied the five stages of design thinking:

1. **Empathise** — studying how freelancers and clients use existing platforms (Chapter 2) and listing the frustrations of both sides, such as bidding fatigue and finding trustworthy partners.
2. **Define** — the core problem statement: *"Freelance developers cannot easily find trusted partners and deliver as a team on the same platform where clients hire them."*
3. **Ideate** — brainstorming produced ideas such as squads, synergy scores, completion-gated ratings and a public leaderboard.
4. **Prototype** — the first clickable prototype was generated with Lovable and then refined by hand in code.
5. **Test** — each sprint build was exercised by the team in both roles; the findings led to changes such as separating client and developer profile titles (commit of 30 Aug 2026) and removing mock data (2 Sep 2026).

3.3.2 Quality Assurance Strategy

- **Static analysis:** the TypeScript compiler (`tsc --noEmit`) and ESLint with React Hooks rules on every significant change.
- **Automated UI tests:** headless Chrome driven by Puppeteer to exercise authentication flows and responsive layout.
- **Security verification:** direct queries with an anonymous Supabase client to confirm that RLS denies private reads and unauthenticated writes.
- **Performance audit:** Google Lighthouse on the production build.
- **Manual acceptance testing:** two-account scenarios for connections, messaging, team batches and ratings (Section 4.2.1).
- **Database test suite:** a Python script runs 104 behaviour and security checks as different users against a local PostgreSQL 16 instance with every migration applied (Chapter 4, Section G).

3.3.3 Tools for Collaboration

Table 3.8: Collaboration and development tools

Tool	Usage
Git and GitHub	Version control; 102 commits; history used to reconstruct sprint dates
Lovable	AI-assisted generation of the initial UI scaffold and quick iterations
Visual Studio Code	Primary code editor with ESLint and Tailwind extensions
Supabase Studio	Database tables, SQL migrations, auth provider and storage configuration
Puppeteer + headless Chrome	Automated UI test cases and screenshots (Chapter 4)
Chrome DevTools / Lighthouse	Debugging, network inspection and performance audits

3.3.4 Project Timeline (Gantt Chart)

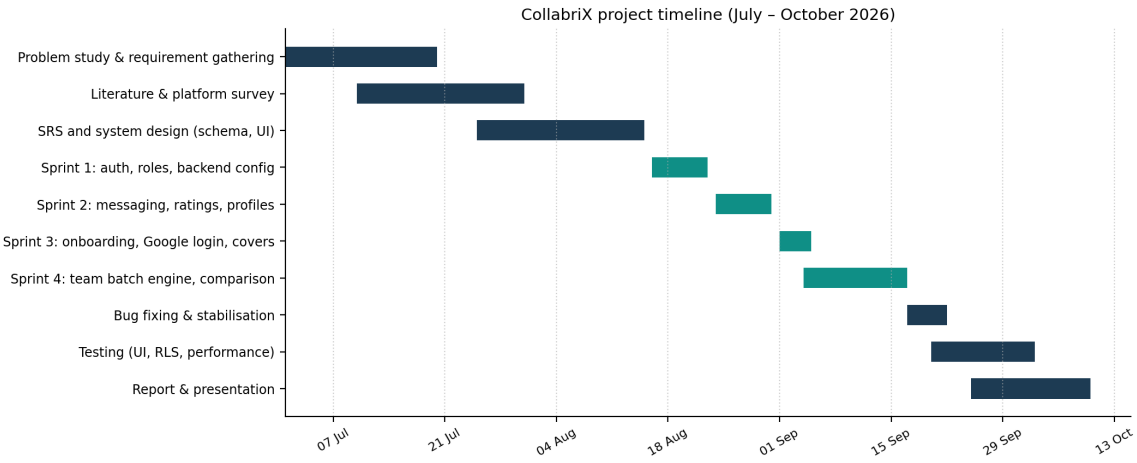


Figure 3.2: Gantt chart of the project timeline

3.4 System Architecture

CollabriX follows a **client–server architecture with a backend-as-a-service (BaaS)**. The client is a single-page application that runs entirely in the browser. Instead of a custom application server, it talks directly to Supabase, which exposes the PostgreSQL database through an auto-generated REST API (PostgREST), authenticates users with JWTs, and broadcasts database changes over WebSockets. Business rules about *who may do what* are implemented as Row-Level Security policies and triggers inside PostgreSQL, so they apply no matter which client sends the request.

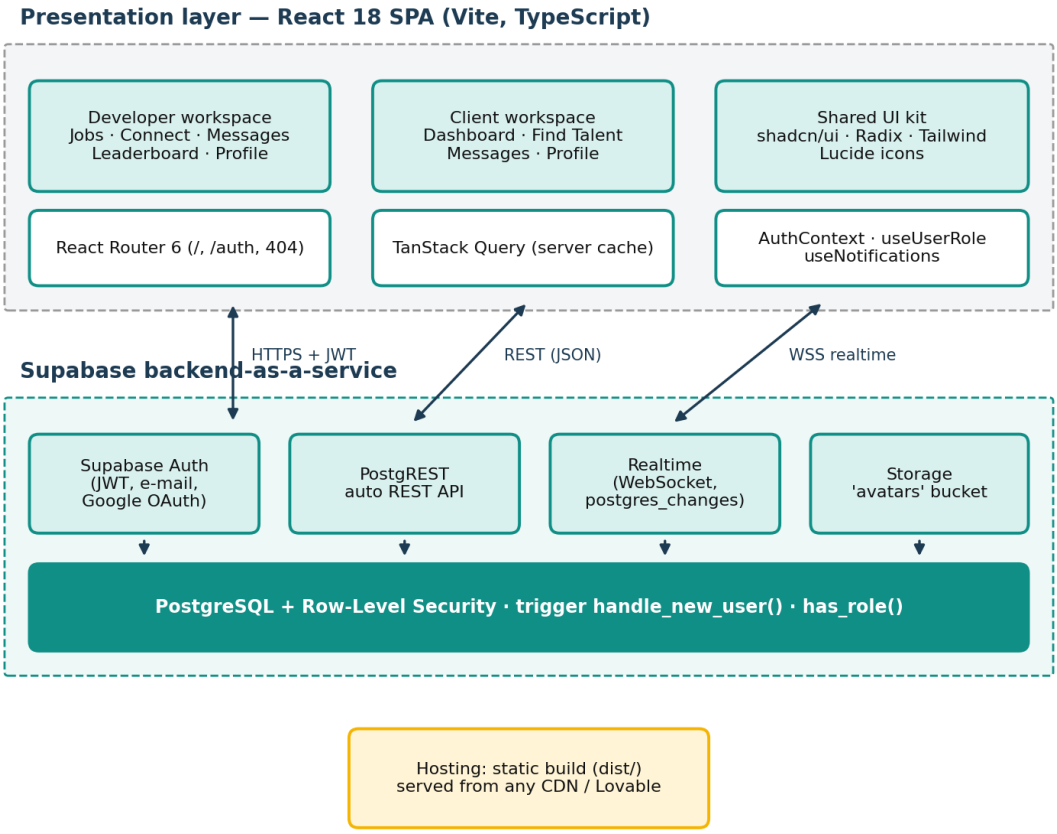


Figure 3.3: Layered system architecture of CollabriX

Table 3.9: Architectural layers and responsibilities

Layer	Components	Responsibility
Presentation	Pages (Index, Auth, Notifications, Settings, Support, Profile view, Project details, NotFound), 10 screens, 21 UI primitives	Rendering, navigation, form validation
State	AuthContext, AppShell, useUserRole, useConnections, useNotificationFeed, useUserSettings, TanStack Query	Session, role, unread counts, cached server data
API	supabase-js → PostgREST, Auth, Realtime, Storage	HTTP and WebSocket communication
Data & security	12 tables, RLS on every table, validation and notification triggers, security-definer functions	Persistence, integrity, authorisation

The extension keeps this layering but moves more responsibility into the data layer. Every workflow that changes the state of a shared object (accepting a request, assigning a developer, completing a project) is either guarded by a trigger or performed by a database function, so a modified client cannot skip a step. The front end calls these functions through PostgREST's remote procedure call (RPC) endpoint, and renders loading, error and empty states based on the real response.

3.5 Database Design

The database consists of twelve application tables in the `public` schema plus the `auth.users` table managed by Supabase. Every profile row shares its primary key with the corresponding `auth.users` row, which makes `auth.uid()` directly comparable with profile identifiers inside RLS policies. The seven original tables are shown in the entity–relationship diagram below; the five tables and the columns added by the collaboration extension are shown in Figure 3.5.

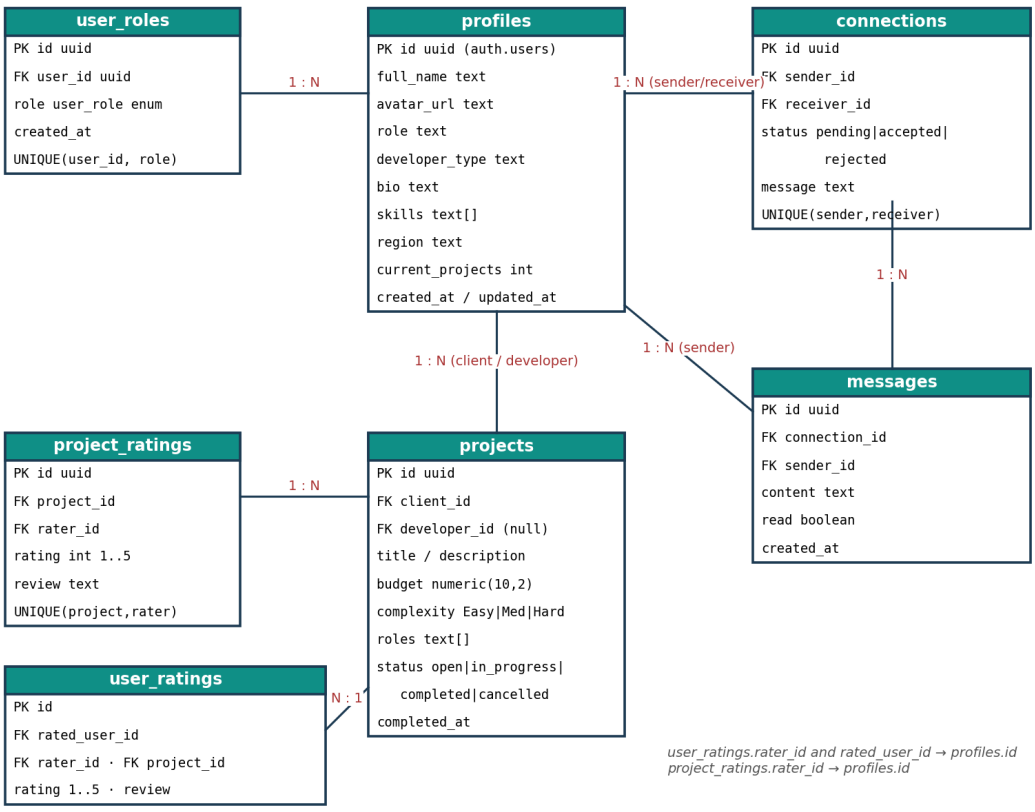


Figure 3.4: Entity–relationship diagram

Table 3.10: Description of database tables

Table	Purpose	Key constraints
profiles	Public identity: name, avatar, bio, role title, developer type, skills, region	PK = auth.users.id (ON DELETE CASCADE)
user_roles	Authorisation role: developer or client (enum user_role)	UNIQUE(user_id, role); one role per user via RLS

Table	Purpose	Key constraints
connections	Connection requests between two profiles	status ∈ {pending, accepted, rejected}; UNIQUE(sender, receiver)
messages	Chat messages inside a connection	FK connection_id; read flag; realtime enabled
projects	Work posted by a client, optionally assigned to a developer	complexity ∈ {Easy, Medium, Hard}; status ∈ {open, in_progress, completed, cancelled}
project_ratings	Rating of a project by a participant	rating 1–5; UNIQUE(project_id, rater_id)
user_ratings	Rating of one participant by another	rating 1–5; UNIQUE(rated_user, rater, project)
team_batches	Saved team batches with members, roles and synergy score	1–10 members, accepted connections only, no duplicates (trigger)
notifications	In-app notifications for one recipient	type ∈ 10 values; rows created only by triggers
project_invitations	A client's invitation of a developer to a project	status ∈ {pending, accepted, declined, cancelled}; UNIQUE(project, developer)
user_settings	Notification preferences, leaderboard visibility, theme	PK = user_id; own row only
support_tickets	Help requests with a public ticket number	category and status CHECKs; subject 5–150, description 20–5000 characters

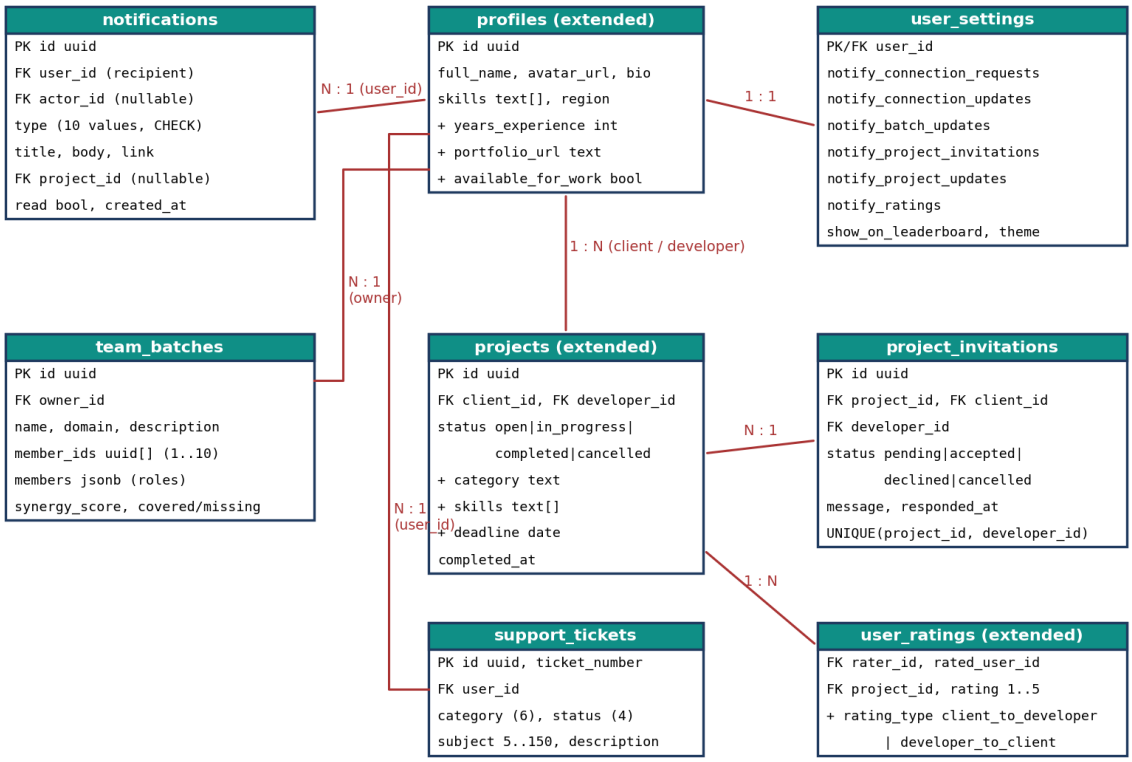
The following table gives the data dictionary of the central `profiles` table, which is read by almost every screen.

Table 3.11: Data dictionary of the profiles table

Column	Type	Default / constraint	Description
id	uuid	PK, FK → auth.users(id) ON DELETE CASCADE	Same identifier as the authentication account
full_name	text	from sign-up metadata, else 'New User'	Person's name, or company name for clients
avatar_url	text	nullable	Public URL of the image in the avatars bucket
role	text	'Full Stack Developer' or 'Client'	Headline shown on cards; used to hide clients from talent search
developer_type	text	'Full Stack Developer'; NULL for clients	Specialisation used by type filters
bio	text	nullable	Free-text professional summary
skills	text[]	'{}'	Skill tags used for search and synergy scoring
region	text	nullable	Continent or country used by the region filter
current_projects	integer	NOT NULL DEFAULT 0	Workload counter reserved for team suggestions

Column	Type	Default / constraint	Description
created_at / updated_at	timestamptz	now(); trigger updates updated_at	Audit timestamps

Figure 3.5 shows how the collaboration extension connects to the existing schema. All new tables reference `profiles` (and therefore `auth.users`) with `ON DELETE CASCADE`, so deleting an account removes its notifications, settings, invitations and tickets automatically.



'+' marks columns added by the collaboration migration; existing tables (connections, messages, user_roles, project_ratings) are unchanged in shape.

Figure 3.5: Entity–relationship diagram of the collaboration extension

The notifications table, which every screen reads through the header bell, is described column by column below.

Table 3.12: Data dictionary of the notifications table

Column	Type	Default / constraint	Description
id	uuid	PK, gen_random_uuid()	Identifier
user_id	uuid	FK → profiles, NOT NULL	Recipient; the only user who can read the row
actor_id	uuid	FK → profiles, nullable	User who caused the event (shown as avatar)
type	text	CHECK: 10 allowed values	Event kind, e.g. connection_request, project_assigned
title / body	text	title NOT NULL	Human-readable message composed by the trigger
link	text	nullable	In-app route opened on click, e.g. /projects/<id>

Column	Type	Default / constraint	Description
entity_id	uuid	nullable	Related connection, batch or project
read	boolean	DEFAULT false	Changed only through mark_notification_read functions
created_at	timestamptz	DEFAULT now()	Ordering of the feed

3.6 UML Design Diagrams

3.6.1 Use Case Diagram

The use case diagram below shows the main use cases. Developers and clients share messaging, profile editing and rating; developer-only use cases cover networking and team formation, while client-only use cases cover company onboarding, talent search and the dashboard.

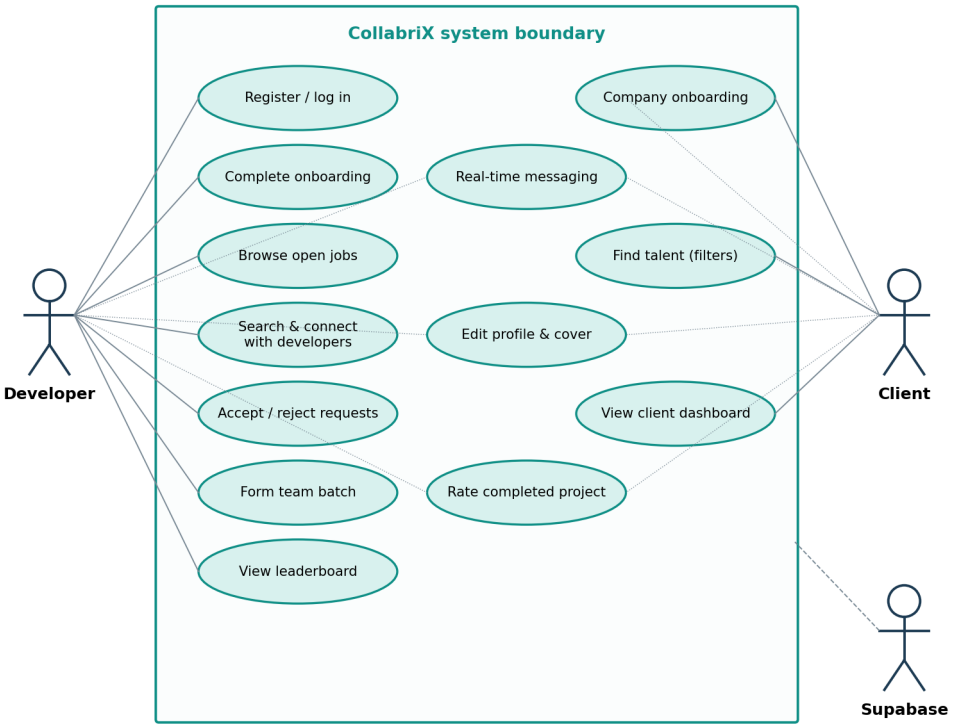


Figure 3.6: Use case diagram

3.6.2 Sequence Diagram — Registration and Onboarding

The sequence diagram below shows the sign-up flow. The role is passed as user metadata so that the database trigger `handle_new_user()` can create both the profile and the role record atomically, before the client even receives the session.

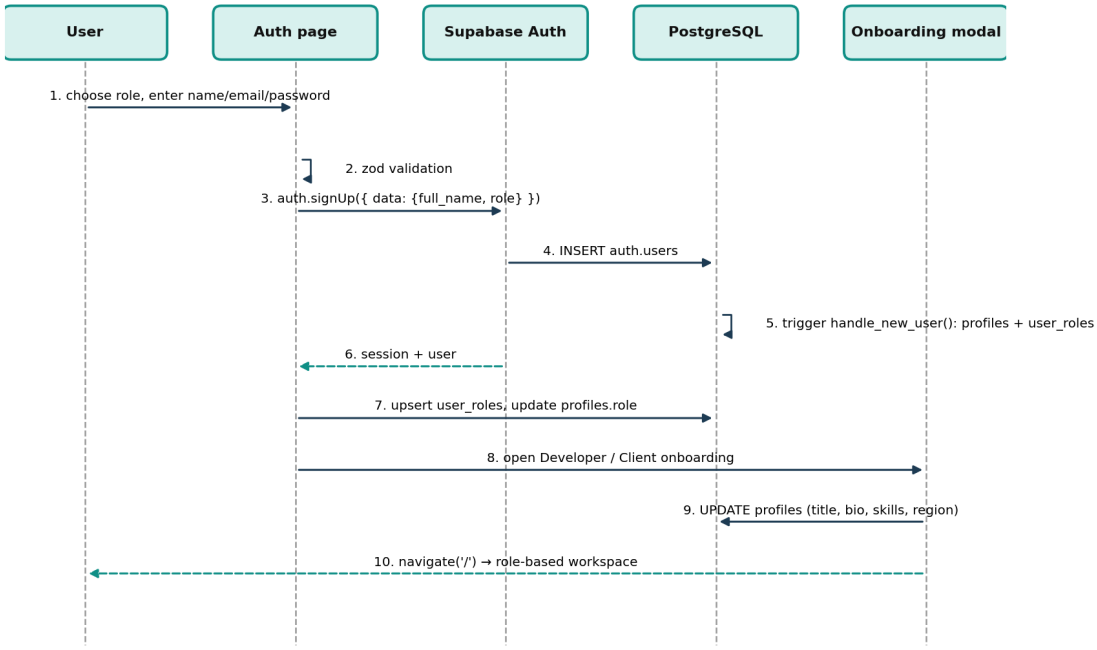


Figure 3.7: Sequence diagram for registration and onboarding

3.6.3 Sequence Diagram — Connection and Real-time Messaging

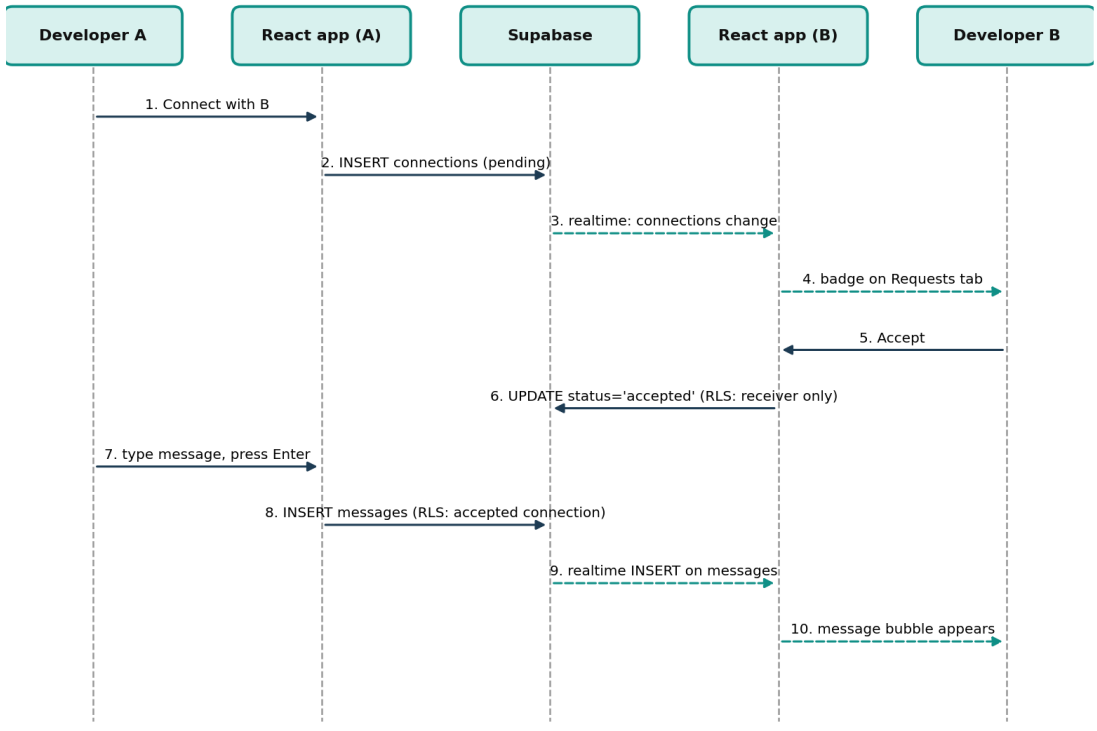


Figure 3.8: Sequence diagram for connection request and messaging

3.6.4 Navigation Design

Navigation is driven by a single route (/) whose content depends on the active tab and the user's role. This keeps transitions instant (no page reloads) and makes the application feel like a native mobile app.

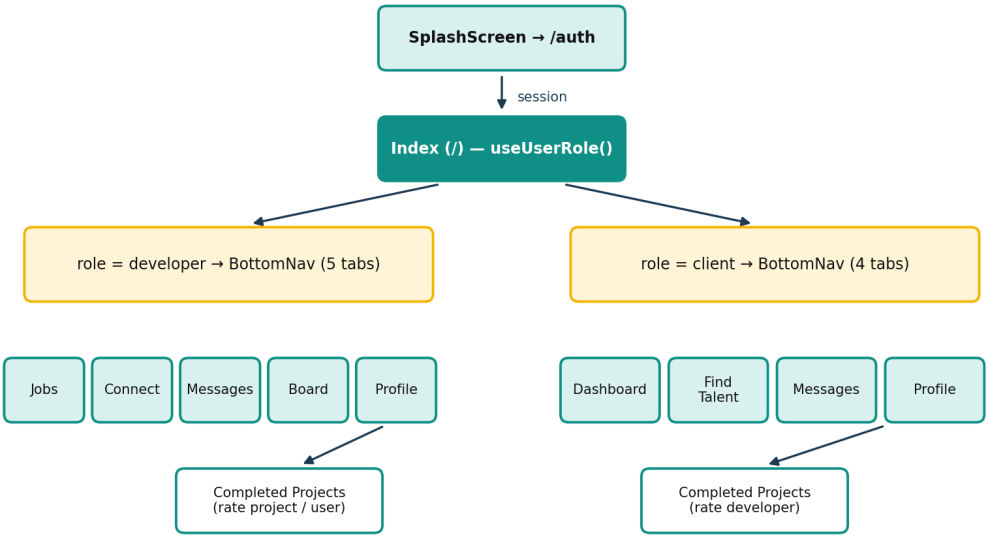


Figure 3.9: Role-based navigation map

The extended version keeps tab-based navigation for the two workspaces but stores the active tab in the URL (`/?tab=connect&view=requests`), so refresh, back/forward and links from notifications work. Five pages that are shared by both roles were added as real routes. All of them are wrapped by the same `AppShell` component, which redirects signed-out visitors to `/auth` and returns them to the requested page after login.

3.6.5 State Diagram — Connection Requests

A connection request is a small state machine. The rules are enforced by two triggers on the `connections` table: `validate_connection_insert` forces every new row to `pending` and blocks requests to oneself and duplicates in the reverse direction, and `validate_connection_update` allows only the receiver to move a pending request to `accepted` or `rejected` and freezes the participants (Listing 3.3).

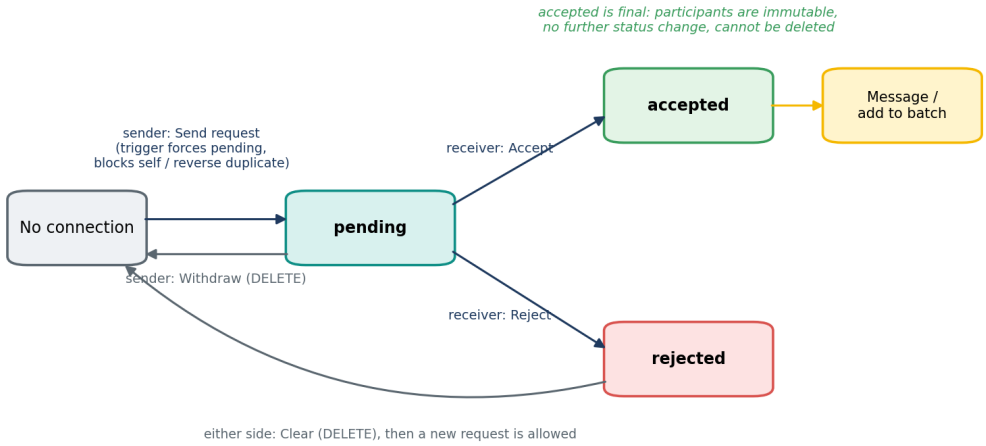


Figure 3.10: State diagram of a connection request

3.6.6 State Diagram — Project and Invitation Lifecycle

Projects and invitations have coupled lifecycles. A project starts `open`; it becomes `in_progress` only when the client assigns a developer who has accepted an invitation, and `completed` only when the client marks it complete, which is the event that unlocks ratings. Invitations move from `pending` to `accepted` or `declined` by the developer's answer, and the remaining pending invitations are `cancelled` automatically when another developer is assigned.

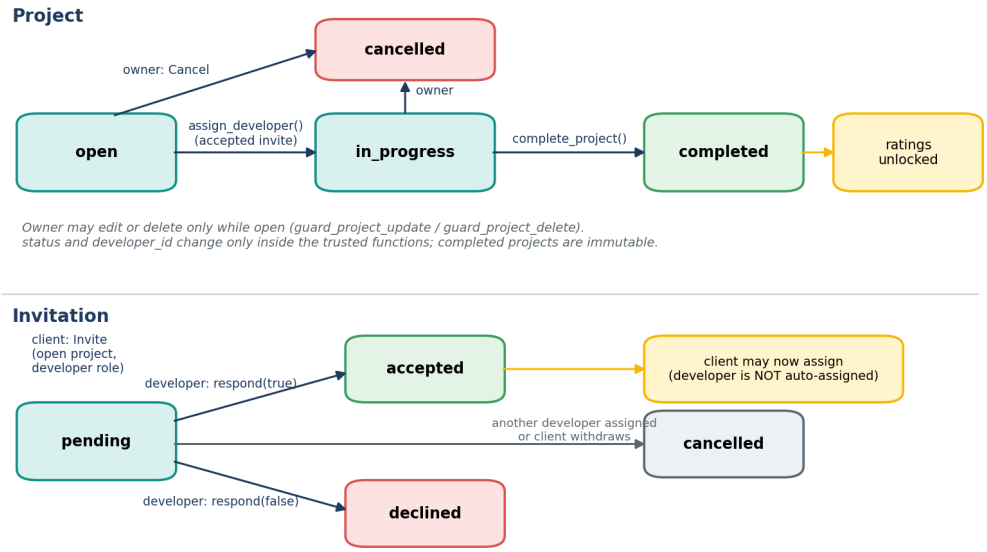


Figure 3.11: State diagrams of the project and invitation lifecycles

3.6.7 Sequence Diagram — From Invitation to Rating

Figure 3.12 follows one engagement from end to end. Dashed arrows are notifications produced by database triggers and delivered over Realtime; every solid arrow towards Supabase is checked by RLS or by a security-definer function before it changes any data.

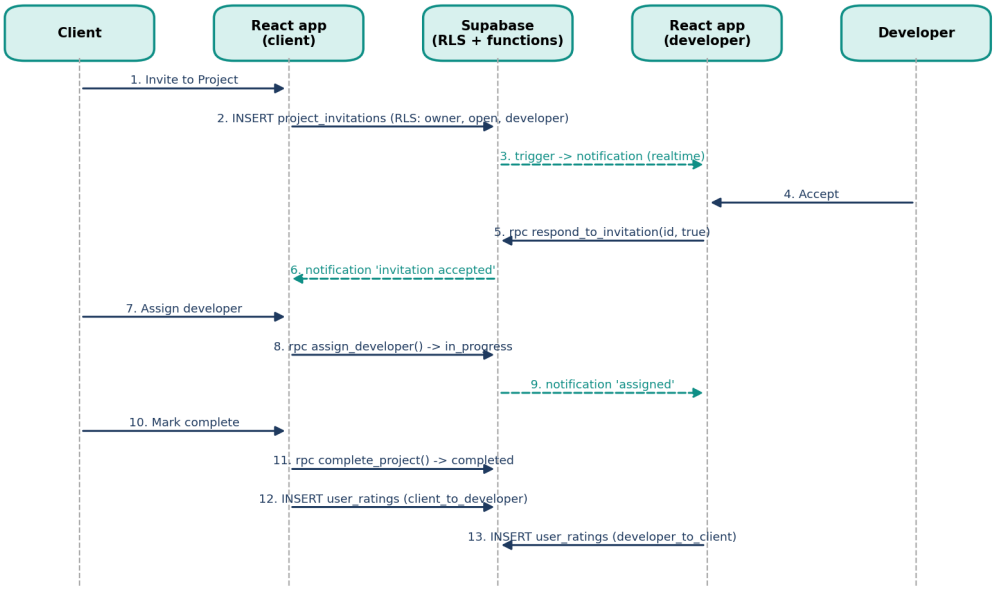


Figure 3.12: Sequence diagram from invitation to two-way rating

3.7 User Interface Design

The interface follows a **mobile-first, component-driven** approach. Design tokens (colours, radii, shadows, gradients) are defined once as CSS custom properties in `index.css` and consumed through Tailwind utility classes, so the whole application can switch between light and dark themes by toggling a single class. The primary colour is teal (HSL 175° 80% 35%), chosen to convey trust and calm. Components such as buttons, dialogs, tabs and selects come from shadcn/ui, which wraps accessible Radix UI primitives — keyboard navigation and ARIA attributes are therefore built in.

- **Consistency:** every screen uses the same card, badge and empty-state patterns.
- **Feedback:** loading spinners for every fetch, and toast messages for every mutation.
- **Progressive disclosure:** filters are hidden behind a filter button; onboarding is split into three short steps.
- **Honest states:** every list has loading, error-with-retry and empty states; destructive actions use an in-app confirmation dialog, and a success message appears only after the database confirms the change.
- **URL-driven navigation:** tabs and sub-views live in the query string, so every screen can be linked to, refreshed and reached from a notification.
- **Accessibility:** icon-only buttons have labels, rows that open a profile are keyboard-focusable, and colour is never the only signal of state.

3.8 Communication Profile

CollabriX uses two communication channels between the browser and the backend. Request–response operations (reading profiles, inserting a message, updating a connection) travel over HTTPS to PostgREST with the user's JWT in the Authorization header [14]. Change notifications travel the other way over a single multiplexed WebSocket connection to

Supabase Realtime [13], on which the client opens one channel per subscription. The following table summarises the channels used.

Table 3.13: Communication channels used by CollabriX

Channel	Protocol	Direction	Used for
PostgREST API	HTTPS + JSON, JWT	Client → server	All reads and writes on application tables
Supabase Auth	HTTPS + JSON	Client ↔ server	Sign-up, login, OAuth redirect, token refresh
Realtime: messages-<id>	WebSocket	Server → client	New messages in an open chat thread
Realtime: connection-requests	WebSocket	Server → client	New or updated requests for the receiver
Realtime: notifications-*	WebSocket	Server → client	Unread badge counts in header and navigation
Storage API	HTTPS (multipart)	Client → server	Avatar upload; public URL for display
RPC functions	HTTPS + JSON, JWT	Client → server	respond_to_invitation, assign_developer, complete_project, get_leaderboard, mark_*_read
Realtime: notifications-feed	WebSocket	Server → client	New notifications for the signed-in user
Realtime: my-connections	WebSocket	Server → client	Requests accepted, rejected or withdrawn on either side

- **Caching:** TanStack Query caches project and rating queries and invalidates them after mutations, avoiding repeated network calls.
- **Realtime instead of polling:** Supabase channels subscribe to `postgres_changes` filtered by connection or receiver, so only relevant events reach each client.
- **Embedded joins:** a single PostgREST request fetches a project together with its client, developer and ratings using foreign-key embedding.
- **Integration:** Google OAuth via Supabase Auth; Supabase Storage for avatars; environment variables (`VITE_SUPABASE_URL`, `VITE_SUPABASE_PUBLISHABLE_KEY`) for configuration.

3.9 Security Design

Security is designed on a zero-trust principle: the browser holds only a public (publishable) key, and the database decides on every row. The following table lists the policies implemented in the migrations.

Table 3.14: Row-Level Security policies implemented

Table	Operation	Policy (who is allowed)
profiles	SELECT / INSERT / UPDATE	Everyone may read; a user may insert and update only the row where id = auth.uid()

Table	Operation	Policy (who is allowed)
user_roles	SELECT / INSERT	Read own role; insert only developer/client and only if the user has no role yet
connections	SELECT / INSERT / UPDATE / DELETE	Parties may read; sender inserts (always pending); only the receiver answers a pending request; sender may withdraw a pending request; accepted connections are permanent
messages	SELECT / INSERT / UPDATE	Only members of an accepted connection may read or send; sender may update own messages
projects	SELECT / INSERT / UPDATE / DELETE	Anyone may read; only users with the client role create (status open); owner edits or deletes while open; status and assignment change only through functions
project_ratings	SELECT / INSERT	Anyone may read; insert only by a participant of a completed project
user_ratings	SELECT / INSERT	Only the counterpart of a completed project; direction recorded by trigger; ratings cannot be edited
team_batches	SELECT / INSERT / UPDATE / DELETE	Owner and members may read; owner writes; trigger accepts only 1–10 accepted connections
notifications	SELECT / DELETE	Recipient only; no INSERT or UPDATE policy, so rows come only from triggers
project_invitations	SELECT / INSERT / DELETE	Client and developer read; owner of an open project invites a developer; client withdraws a pending invitation
user_settings	SELECT / INSERT / UPDATE	Own row only
support_tickets	SELECT / INSERT	Own tickets only; new tickets must be open; status changed by administrators
storage: avatars	INSERT / UPDATE / DELETE	Only inside the folder named after the user's own id

Additional measures include input validation with Zod on the client, parameterised queries through PostgREST (eliminating SQL injection), React's automatic output escaping (mitigating XSS), and `SECURITY DEFINER` functions with a fixed `search_path` to prevent search-path hijacking.

The back end is defined entirely in SQL migrations, which makes it reproducible on any Supabase project. The most important piece is the trigger that runs when a new user is inserted into `auth.users`. It reads the role from the sign-up metadata and provisions both the profile and the role record:

Listing 3.1: Sign-up trigger (abridged)

```
CREATE OR REPLACE FUNCTION public.handle_new_user()
RETURNS TRIGGER LANGUAGE plpgsql SECURITY DEFINER SET search_path = public AS $$
DECLARE user_role_val public.user_role; raw_role text;
BEGIN
  raw_role := NEW.raw_user_meta_data ->> 'role';
  IF raw_role = 'client' THEN user_role_val := 'client';
  ELSE user_role_val := 'developer'; END IF;
  INSERT INTO public.profiles (id, full_name, avatar_url, role, developer_type)
  VALUES (NEW.id, COALESCE(NEW.raw_user_meta_data ->> 'full_name', 'New User'), ...)
  ON CONFLICT (id) DO UPDATE SET full_name = EXCLUDED.full_name, ...;
  INSERT INTO public.user_roles (user_id, role) VALUES (NEW.id, user_role_val)
  ON CONFLICT (user_id, role) DO NOTHING;
  RETURN NEW;
END; $$;
```

Ratings are protected by a policy that checks project participation and completion status in the same statement:

Listing 3.2: Completion-gated rating policy

```
CREATE POLICY "Users can rate others from completed projects"
ON public.user_ratings FOR INSERT WITH CHECK (
  auth.uid() = rater_id AND auth.uid() != rated_user_id AND
  EXISTS (SELECT 1 FROM public.projects p
          WHERE p.id = project_id
            AND (p.client_id = auth.uid() OR p.developer_id = auth.uid())
            AND p.status = 'completed'));
```

The `messages` and `connections` tables are added to the `supabase_realtime` publication so that inserts and updates are broadcast to subscribed clients, and an `updated_at` trigger keeps modification timestamps accurate on profiles, connections and projects.

3.9.1 Workflow Rules Enforced by Triggers

RLS answers the question *who may touch this row*, but not *which change is allowed*. For example, the receiver of a connection request is allowed to update the row, yet must not be able to rewrite the sender or turn an accepted connection back into a rejected one. CollabriX therefore pairs RLS with `BEFORE` triggers that validate the change itself. Listing 3.3 shows the trigger for connection updates.

Listing 3.3: Validation trigger for connection updates (abridged)

```
CREATE FUNCTION public.validate_connection_update() RETURNS TRIGGER AS $$
BEGIN
  IF NEW.sender_id <> OLD.sender_id OR NEW.receiver_id <> OLD.receiver_id THEN
    RAISE EXCEPTION 'Connection participants cannot be changed'
      USING ERRCODE = 'insufficient_privilege';
  END IF;
  IF NEW.status <> OLD.status THEN
    IF OLD.status <> 'pending' OR NEW.status NOT IN ('accepted','rejected') THEN
      RAISE EXCEPTION 'Invalid connection status change from % to %',
        OLD.status, NEW.status USING ERRCODE = 'check_violation';
    END IF;
    IF auth.uid() IS NOT NULL AND auth.uid() <> OLD.receiver_id THEN
      RAISE EXCEPTION 'Only the receiver can respond to a connection request';
    END IF;
  END IF;
  RETURN NEW;
END; $$ LANGUAGE plpgsql;
```

The same pattern protects the other tables: `validate_team_batch` checks the size and membership of a batch, `guard_project_update` and `guard_project_delete` stop owners from changing assignment, completion or finished projects, and `prepare_user_rating` derives the rating direction from the project instead of trusting the client.

3.9.2 Trusted Operations and Security-Definer Functions

Some changes are legitimate only as part of a larger operation. A client may not set `developer_id` on a project directly, but assigning a developer who accepted an invitation must do exactly that. These operations are implemented as `SECURITY DEFINER` functions that first check the caller with `auth.uid()`, then set a transaction-local flag (`collabrix.trusted`) that the guard trigger recognises, and finally make the change. Because the flag is set with `set_config(..., true)`, it disappears at the end of the transaction and cannot be set by a normal API request. Table 3.15 lists the functions and Listing 3.4 shows the most important one.

Table 3.15: Database functions exposed to or used by the application

Function	Caller	What it checks / does
respond_to_invitation(id, accept)	Invited developer	Invitation is theirs and still pending; records the answer; notifies the client
assign_developer(project, developer)	Project owner	Project is open and unassigned; developer accepted an invitation; starts the project; cancels other pending invitations
complete_project(project)	Project owner	Project is in progress; sets completed and completed_at; notifies the developer
get_leaderboard(since)	Signed-in users	Computes scores from verified completed projects and client ratings (Algorithm 2)
mark_notification_read(id)	Recipient	Marks one of the caller's own notifications as read
mark_all_notifications_read()	Recipient	Marks all of the caller's notifications as read; returns the count
mark_messages_read(connection)	Connection party	Marks the other party's messages in that conversation as read
create_notification(...)	Triggers only	Respects the recipient's preferences; never notifies users about their own actions; EXECUTE revoked from clients

Listing 3.4: Assigning a developer only after an accepted invitation (abridged)

```
CREATE FUNCTION public.assign_developer(_project_id UUID, _developer_id UUID)
RETURNS void LANGUAGE plpgsql SECURITY DEFINER SET search_path = public AS $$
DECLARE proj public.projects%ROWTYPE;
BEGIN
  SELECT * INTO proj FROM projects WHERE id = _project_id FOR UPDATE;
  IF NOT FOUND OR proj.client_id <> auth.uid() THEN
    RAISE EXCEPTION 'Project not found'; END IF;
  IF proj.status <> 'open' OR proj.developer_id IS NOT NULL THEN
    RAISE EXCEPTION 'Only open, unassigned projects can be assigned'; END IF;
  IF NOT EXISTS (SELECT 1 FROM project_invitations i WHERE i.project_id =
_project_id
      AND i.developer_id = _developer_id AND i.status = 'accepted') THEN
    RAISE EXCEPTION 'You can only assign a developer who accepted your invitation';
  END IF;
  PERFORM set_config('collabrix.trusted', 'on', true);
  UPDATE projects SET developer_id = _developer_id, status = 'in_progress'
   WHERE id = _project_id;
  UPDATE project_invitations SET status = 'cancelled'
   WHERE project_id = _project_id AND developer_id <> _developer_id AND status =
'pending';
  PERFORM set_config('collabrix.trusted', 'off', true);
  PERFORM create_notification(_developer_id, proj.client_id,
'project_assigned', ...);
END; $$;
```

The row is locked with `SELECT ... FOR UPDATE`, so two concurrent assignments cannot both succeed, and the whole function runs in one transaction: either the project starts, the other invitations are cancelled and the developer is notified, or nothing changes.

3.9.3 Threat Model Review

Table 3.16 reviews the main abuse cases of the extended platform and where each one is stopped. Each row marked *tested* is covered by the database test suite in Chapter 4.

Table 3.16: Threats considered and their mitigations

Threat	Example	Mitigation
Forged notifications	User inserts a fake “rating received” notification for someone else	No INSERT policy on notifications; create_notification revoked from clients (tested)
Self-assignment	Developer sets developer_id on a project	guard_project_update rejects direct changes; only assign_developer may assign (tested)
Assignment without consent	Client assigns a developer who never accepted	assign_developer requires an accepted invitation (tested)
Rating abuse	Rating without working together, rating twice, rating oneself	Insert policy requires the counterpart of a completed project; UNIQUE constraint (tested)
Request spoofing	Sending a request on someone else's behalf, accepting one's own request	RLS on sender_id; trigger allows only the receiver to answer (tested)
Data exposure	Reading other users' tickets, invitations or notifications	Row-owner SELECT policies; anonymous reads return no rows (tested)
Privilege escalation	Inserting an admin role for oneself	user_roles insert limited to developer/client and only if no role exists
Secret leakage	Service-role key shipped to the browser	Only the publishable key is bundled; build scanned for service_role

Threat	Example	Mitigation
Cross-site scripting	Script in a project description or message	React escapes output; no dangerouslySetInnerHTML for user content

3.10 Module Implementation

3.10.1 Project Structure

The source code contains approximately 14,200 lines of TypeScript/TSX in 86 files in the `src` folder (up from about 9,700 before the extension) and eleven SQL migration files. The following table summarises the main folders.

Table 3.17: Source code organisation

Folder	Contents
src/pages	Index (role-based shell), Auth, NotificationsPage, SettingsPage, SupportPage, ProfileViewPage, ProjectDetailsPage, NotFound
src/screens	JobsScreen, ConnectScreen, MessagesScreen, LeaderboardScreen, ProfileScreen, ClientDashboard, ClientFindTalentScreen, ClientMessagesScreen, ClientProfileScreen, CompletedProjectsScreen
src/components	auth (onboarding), connect (ConnectionAction, search, requests, connections, message thread, team engine), layout (AppShell, Header, BottomNav), projects (PostProjectDialog, InviteToProjectDialog), gamification, cards, ratings, profile, filters, ConfirmDialog, ui (21 primitives)
src/hooks	useAuth, useUserRole, useProjects, useConnections, useNotificationFeed, useNotifications, useUserSettings, use-mobile, use-toast
src/lib	utils (class names) and errors (friendlyError: maps database errors to clear messages)
src/integrations/supabase	Typed Supabase client and generated database types
supabase/migrations	11 SQL migrations: tables, enums, RLS policies, storage bucket, sign-up trigger, team batches, read receipts and the collaboration core (5 tables, 13 triggers, 19 functions)

3.10.2 Authentication and Role Routing

`AuthProvider` subscribes to `onAuthStateChange` before reading the existing session, so no auth event is missed during start-up. Network failures are translated into a clear message that points to the `VITE_SUPABASE_URL` setting. The `Index` page waits for both the session and the role to load, redirects anonymous users to `/auth`, and renders either the developer or the client screen set. The flowchart below shows the logic.

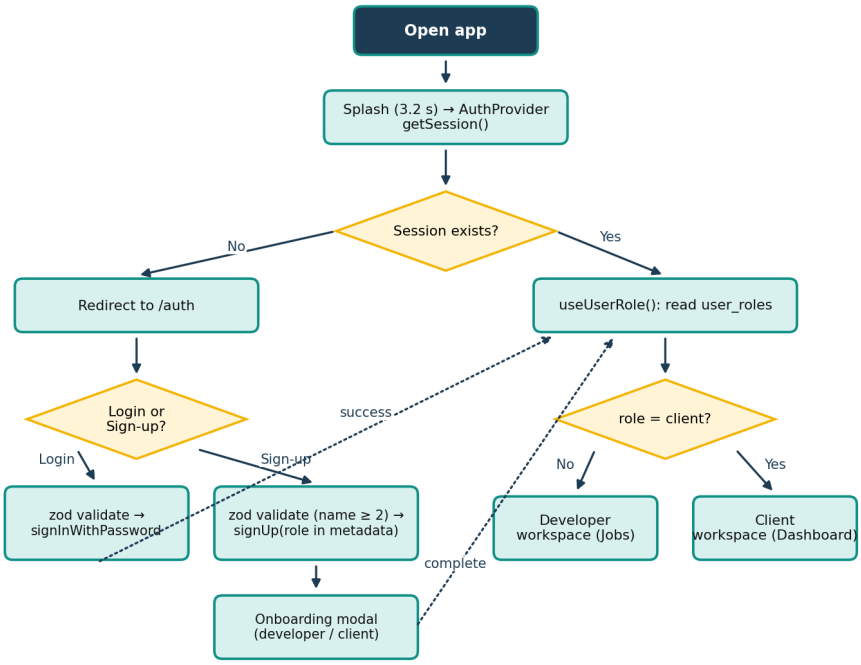


Figure 3.13: Flowchart of authentication and role-based routing

Form inputs are validated with Zod schemas before any network call: e-mail must be well-formed, the password must be non-empty and the full name must have at least two characters.

In the extended version, routing for every signed-in page is centralised in an `AppShell` component. It waits for the session and the role, redirects signed-out visitors to `/auth` while remembering the requested address, and renders the header and bottom navigation. `useUserRole` no longer guesses a role when the network fails; it shows an error with a retry button instead, and inserts the default developer role only when no role row exists. When a Google OAuth 2.0 sign-in [26] fails, Supabase returns the reason in the redirect URL; the application captures it before the redirect to `/auth` and shows it to the user instead of silently returning to the sign-in form.

3.10.3 Onboarding and Profiles

After registration, a three-step modal opens. The **developer** wizard offers three paths (LinkedIn link, résumé upload or manual entry) and finally shows a live preview of the profile card before saving title, bio and skills. The **client** wizard captures company name, website, location, industry, company size, hiring goal and mission statement. The developer profile screen shows avatar, skills, connection count, completed projects, rating summary and six dynamically earned badges; avatars are uploaded to the `avatars` storage bucket under the user's own folder.

The onboarding was also made honest. The LinkedIn and résumé paths previously displayed a success message without extracting any data; they now open the manual form with an

explanation. The developer form validates the title and years of experience and stores `years_experience` in the profile, and both wizards report a real error if saving fails.

3.10.4 Developer Discovery, Connections and Requests

The Connect screen has four tabs: *Find Developers*, *My Connections*, *Team Engine* and *Requests*. Search combines free-text matching on name, type and skills with select-based filters for developer type, skill and a two-level region filter (continent → country). Each developer card shows the current connection state — *Connect*, *Request Sent* or *Message* — derived from the `connections` table. Incoming requests are loaded for the receiver and refreshed through a realtime subscription.

All buttons that relate to another person are rendered by one shared `ConnectionAction` component backed by the `useConnections` hook, which loads the user's connections once and keeps them current through a Realtime subscription. Table 3.18 lists the seven relationship states it distinguishes. The Requests tab now shows *Received* requests with Accept and Reject and *Sent* requests with Withdraw, and every name opens the person's public profile.

Table 3.18: Relationship states and the actions offered

State	Meaning	Buttons shown
self	The profile is the current user	None
none	No connection row exists	Send Connection Request
pending_sent	Current user sent a request	Request sent (disabled), Withdraw
pending_received	The other user sent a request	Accept, Reject
accepted	Both are connected	Message, Form Batch (or Connected)
rejected_sent	The other user declined	Request declined (disabled), Clear
rejected_received	Current user declined earlier	Send Connection Request (old row is cleared first)

3.10.5 Real-time Messaging

Each conversation is a `MessageThread` bound to one accepted connection. It loads history ordered by time and subscribes to inserts for that connection only:

Listing 3.5: Realtime subscription in MessageThread

```
const channel = supabase
  .channel(`messages-${connectionId}`)
  .on('postgres_changes',
      { event: 'INSERT', schema: 'public', table: 'messages',
        filter: `connection_id=eq.${connectionId}` },
      (payload) => setMessages(prev => [...prev, payload.new as Message]))
  .subscribe();
return () => { supabase.removeChannel(channel); };
```

The inbox (`MessagesScreen`) lists all accepted connections with the latest message, time and unread count, sorted by recency, and is shared by both roles.

In the original version the `read` flag of a message was never set, because RLS allowed only the sender to update a message, so unread counts never decreased. The function

`mark_messages_read(connection)` now marks the other party's messages as read when a conversation is opened, and the badges in the navigation update in real time.

3.10.6 Team Formation Engine

Developers create a *team batch* by giving it a name, a project domain and an objective, and by adding members **only from accepted connections**. Each member can be assigned a project role. Batches are stored in the `team_batches` table rather than in the browser, so members see the batches they belong to on any device and are notified when they are added. A validation trigger accepts between one and ten members, rejects the owner, duplicates and anyone who is not an accepted connection, and batches saved in the browser by older versions are imported once automatically. In the Add People dialog, other developers can be found by name and sent a connection request directly, but they can only be selected after accepting. As members are added, the engine compares the union of their skills with a recommended skill set for the selected domain and computes a synergy score using Algorithm 1.

Algorithm 1 Synergy Score Calculation for a Team Batch

Input: selected members M, project domain d
R ← DOMAIN_RECOMMENDED_SKILLS[d] ▷ default: Fullstack Web App
S ← ∪ { skills(m) : m ∈ M } ▷ combined stack
covered ← ∅; missing ← ∅
for each skill r ∈ R **do**
  if ∃ s ∈ S : lower(s) = lower(r) or lower(s) contains lower(r) **then** covered ← covered ∪ {r}
  else missing ← missing ∪ {r}
ratio ← |covered| / |R|
bonus ← min(10 × |M|, 30) ▷ team-size bonus
score ← min(round(70 × ratio + bonus), 100); **if** |M| = 0 **then** score ← 0
return score, S, missing

For example, for the *Fullstack Web App* domain (React, TypeScript, Node.js, PostgreSQL, UI/UX Design, AWS), a two-member team covering four of the six skills scores round(70 × 4/6 + 20) = 67 %. Missing skills are shown as *skill gaps to consider*, guiding the developer to invite a complementary connection.

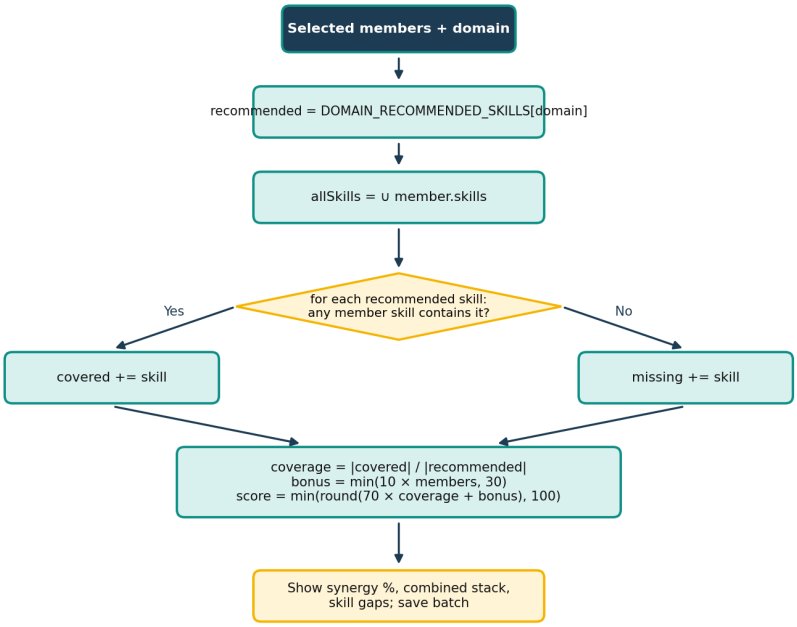


Figure 3.14: Flowchart of the synergy score algorithm

3.10.7 Projects, Ratings and Leaderboard

The `useProjects` hook fetches all projects, the user's own projects and completed projects using embedded joins to profiles and ratings. On the Completed Projects screen each participant sees *Rate Project* and a rating button for the other party (*Rate Developer* for the client, *Rate Client* for the developer). The screen checks which ratings the current user has already given (`useMyGivenRatings`), so each button disappears once used and shows the stars given instead. The rating dialog stays open with the review intact if saving fails. The completion-gated rating policy shown in the Security Design section makes the final decision, as the following flowchart shows.

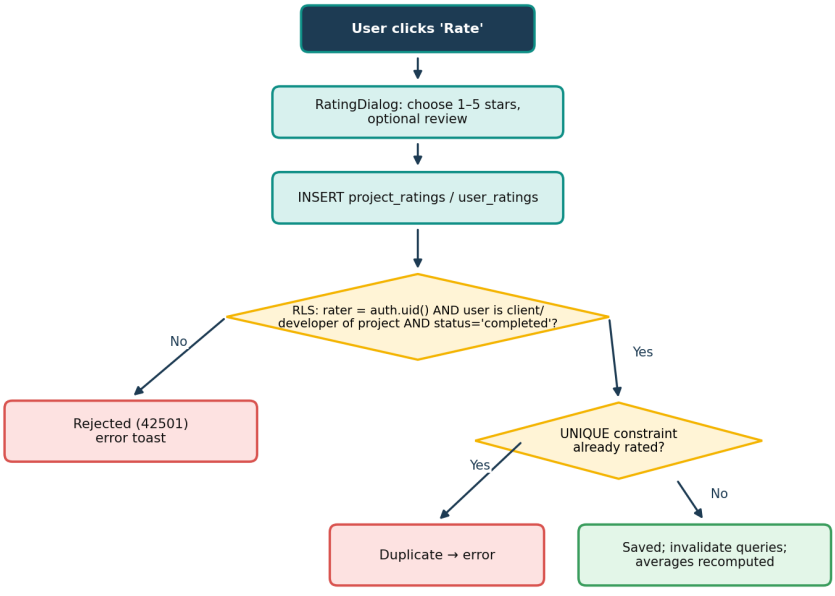


Figure 3.15: Flowchart of rating submission and validation

The leaderboard (Algorithm 2) is computed by the database function `get_leaderboard`, so every viewer sees the same verified numbers. A developer earns 100 points for each project a client marked complete and 20 points per star of each client rating on such a project; a completed five-star project is therefore worth 200 points. Developers without verified work are not listed (the earlier baseline average of 4.5 for unrated profiles was removed) and anyone can hide themselves in Settings. *This Week* and *This Month* pass a period start to the same function. Level tiers and six achievement badges are derived from the score, the average rating, completed projects and connections, and a *How scores work* panel explains the formula on screen.

Algorithm 2 Leaderboard Ranking (database function get_leaderboard)

Input: optional period start t_0 (NULL means all time)
$D \leftarrow$ profiles with role developer whose settings allow show_on_leaderboard
for each developer $d \in D$ **do**
  $C(d) \leftarrow |\{ p : p.\text{developer} = d,\ p.\text{status} = \text{completed},\ p.\text{completed_at} \geq t_0 \}|$
  $R(d) \leftarrow$ client ratings of d on completed projects, created after t_0
  $\text{score}(d) \leftarrow 100 \times C(d) + 20 \times \Sigma$ rating over $R(d)$
  $\text{avg}(d) \leftarrow$ mean rating over $R(d)$, 0 if empty; $n(d) \leftarrow |R(d)|$
end for
$L \leftarrow \{ d \in D : C(d) > 0 \text{ or } n(d) > 0 \}$ ▷ no invented baseline
sort L by score ↓, avg ↓, n ↓, full_name ↑, id ↑ ▷ deterministic tie-breaks
$\text{rank}(L[i]) \leftarrow i$ for $i = 1 \ldots |L|$
return L

Table 3.19 works through the formula for six developers. It also shows a deliberate trade-off: volume of verified work counts, so Chen, with three completed projects and modest ratings, ranks above Asha, who has two excellent ones. Dina and Eli are tied on every numeric criterion and are ordered by name, and Farah, who has no verified work yet, is not listed.

Table 3.19: Worked example of the leaderboard formula

Developer	Completed	Client ratings	Score	Average	Rank
Chen	3	2, 2, 2	300 + 120 = 420	2.0	1
Asha	2	5, 5	200 + 200 = 400	5.0	2
Ben	2	5, 3	200 + 160 = 360	4.0	3
Dina	1	4	100 + 80 = 180	4.0	4 (name)
Eli	1	4	100 + 80 = 180	4.0	5 (name)
Farah	0	—	0	—	not listed

3.10.8 Project Posting and Management

Clients post projects through a validated dialog whose rules mirror the database constraints (Table 3.20). The submit button is disabled while the request is in flight, which prevents double submission, and the dialog stays open with the entered data if the server rejects the project. The same dialog is reused for editing. On the project details page (`/projects/:id`) the owner can edit, cancel or delete an open project, invite developers, assign one who

accepted, withdraw pending invitations and mark an in-progress project complete; every destructive step asks for confirmation first.

Table 3.20: Validation rules of the Post Project form

Field	Rule	Enforced by
Title	5–120 characters	Form (zod) and CHECK constraint
Description	20–4000 characters	Form and CHECK constraint
Category	One of 9 categories	Form
Required skills	1–15 comma-separated skills, duplicates removed	Form
Budget	Optional; greater than 0 and at most 99,999,999	Form and DECIMAL(10,2) column
Deadline	Optional; not in the past (an unchanged past deadline is allowed when editing)	Form
Complexity	Easy, Medium or Hard	Form and CHECK constraint
Status	Always open on creation	RLS insert policy
Owner	client_id = auth.uid() and the user has the client role	RLS insert policy

3.10.9 Invitations and Assignment

The Find Talent screen and every public profile offer *Invite to Project*. The invitation dialog lists only the client's open, unassigned projects and marks those to which the developer is already invited. The developer sees the invitation in the Jobs screen (*Invitations* tab) and in the notification feed and answers it there or on the project page. Accepting does not assign the developer: the client receives a notification and decides whom to assign, as summarised in Algorithm 3.

Algorithm 3 Invitation-gated assignment of a developer

Input: project p, developer d, caller u
require u = p.client and p.status = open and p.developer = NULL
require ∃ invitation i : i.project = p ∧ i.developer = d ∧ i.status = accepted
lock p (SELECT … FOR UPDATE)
p.developer ← d; p.status ← in_progress ▷ inside a trusted transaction
for each invitation j of p with j.developer ≠ d and j.status = pending **do** j.status ← cancelled
notify d: "You were assigned to p"
return

3.10.10 Notification System

Notifications are produced exclusively by the database. Each relevant table has an `AFTER` trigger that calls `create_notification`, which checks the recipient's preference in `user_settings`, skips events caused by the recipient themselves, and inserts the row (Figure 3.16). Because clients cannot insert into the table or call the function, a notification is always evidence of a real event. On the client, `useNotificationFeed` loads the feed with the actor's name and avatar, subscribes to inserts for the current user, and exposes *mark read*, *mark all read* and *delete*. The header bell shows the unread count, and clicking a notification marks it read and opens its link.

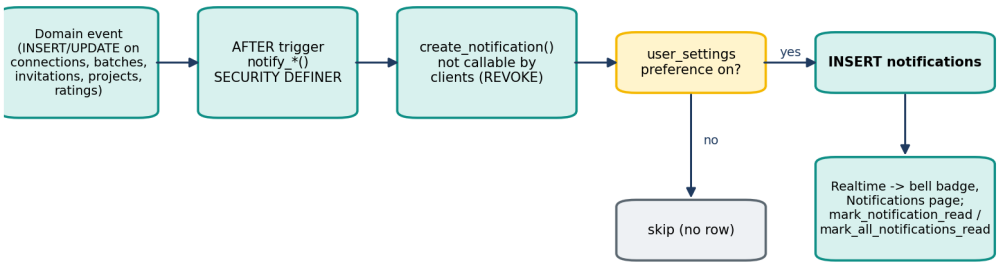


Figure 3.16: Notification pipeline from database event to the user interface

Table 3.21: Notification types, their triggers and the preference that controls them

Type	Created when	Recipient	Preference
connection_request	A request is sent	Receiver	Connection requests
connection_accepted	The receiver accepts	Sender	Connection updates
connection_rejected	The receiver rejects	Sender	Connection updates
batch_added	A user is added to a team batch	Each new member	Batch updates
project_invitation	A client invites a developer	Developer	Project invitations
invitation_accepted	The developer accepts	Client	Project invitations
invitation_declined	The developer declines	Client	Project invitations
project_assigned	The client assigns the developer	Developer	Project updates
project_completed	The client marks the project complete	Developer	Project updates
rating_received	A rating is submitted	Rated user	Ratings

3.10.11 Settings and Account Management

The Settings page (`/settings`) has four tabs that can be linked to directly. *Profile* edits the public profile, including years of experience, portfolio URL and availability for work, which the client's talent filters use. *Notifications* switches each of the six notification groups on or off. *Privacy* controls whether the user appears on the leaderboard. *Account* changes the password (with confirmation), changes the e-mail address through Supabase's confirmation e-mail, signs out, or signs out of every device. Settings are saved with an upsert on the user's own row, and the theme choice follows the user across devices.

3.10.12 Contact Support

Users open support tickets from the account menu or the client profile. A ticket has a subject, one of six categories (account, technical, project, safety, feedback, other) and a description; the form applies the same length limits as the table constraints. Each ticket receives a sequential number from an identity column, starts in status *open* and is listed with its current status (open, in progress, resolved or closed). Only the author can read a ticket; status changes are reserved for administrators.

3.10.13 Error Handling and User Feedback

A common defect of the earlier version was optimistic feedback: some actions displayed success before, or regardless of, the server's answer. All mutations now wait for the result and translate failures through a single helper, `friendlyError`, shown in Table 3.22. Messages raised by the project's own triggers and functions are already written for users and are passed through unchanged.

Table 3.22: Mapping of technical errors to user-facing messages

Technical cause	Detected by	Message shown
RLS or privilege violation (42501)	Error code or “row-level security” text	You don't have permission to do that.
Unique violation (23505)	Error code or “duplicate key” text	This already exists. (specific text for duplicate ratings and requests)
Network failure	“Failed to fetch” / “network”	Network error. Check your connection and try again.
Missing migration	“function … does not exist”, “relation … does not exist”	This feature needs the latest database migration.
Workflow rule	Exception raised by a trigger or function	The function's own message, e.g. “You can only assign a developer who accepted your invitation”

3.11 Configuration and Deployment

The application reads its backend location from two Vite environment variables. To make a fresh clone runnable even without a local `.env` file, `vite.config.ts` injects public fallback values through Vite's `define` option only when a variable is missing:

Listing 3.6: Environment fallback in vite.config.ts

```
const env = loadEnv(mode, process.cwd(), 'VITE_');
const define = Object.fromEntries(
  Object.entries(FALLBACK_ENV)
    .filter(([key]) => !env[key])
    .map(([key, value]) => [`import.meta.env.${key}`, JSON.stringify(value)]));
```

Only the publishable (anonymous) key is ever shipped to the browser; this is safe because every table is protected by RLS. Deployment consists of three steps:

1. Apply the SQL files in `supabase/migrations` to the Supabase project in order (tables, policies, triggers, functions and storage bucket) and enable the Google provider if required. For an existing project the three newest migrations are also combined into `supabase/APPLY_PENDING_MIGRATIONS.sql`, which runs inside a single transaction and records itself in the migration history.
2. Run `npm install` followed by `npm run build`; Vite produces a static `dist/` folder (index.html, one CSS and one JS bundle, and the logo).
3. Upload `dist/` to any static host or CDN and configure it to serve `index.html` for unknown paths so that client-side routes such as `/auth` work on refresh.

Chapter 4

Results and Analysis

4.1 Tools and Technologies

Table 4.1: Tools and technologies used

Category	Tools & technologies
Front-end	React 18.3 · TypeScript 5.8 · Vite 5.4 · Tailwind CSS 3.4 · shadcn/ui (Radix UI) · Lucide icons · React Router 6 · TanStack Query 5 · Zod 3
Back-end (BaaS)	Supabase: PostgreSQL, Row-Level Security, Auth (e-mail, Google OAuth), Realtime, Storage · supabase-js 2.88
Algorithms	Synergy score for team batches (Algorithm 1) · rating-based leaderboard ranking (Algorithm 2)
Development tools	Visual Studio Code · Git & GitHub · Lovable · Supabase Studio · npm
Testing tools	TypeScript compiler · ESLint · Puppeteer + headless Chrome · Lighthouse 12 · PostgreSQL 16 + psycopg (database test suite)
Communication protocols	HTTPS / REST (PostgREST) · WebSocket (Supabase Realtime) · JWT · OAuth 2.0

4.2 Results

The CollabriX prototype is functional for authentication, onboarding, profiles, developer discovery, connections, real-time messaging, team batch formation, ratings and the leaderboard, and, after the collaboration extension, for project posting, invitations, assignment, completion, notifications, settings and support tickets. The following table contrasts the approach of conventional freelance marketplaces with that of CollabriX, and the screenshots after it show the implemented system.

Table 4.2: Comparative analysis of conventional freelance platforms vs. CollabriX

Criteria	Conventional platforms	CollabriX
Unit of hiring	Individual freelancer	Individual or team batch
Peer discovery	Limited or absent	Search + connect with filters
Team composition insight	Manual judgement	Synergy score and skill gaps
Rating integrity	Platform policy	Enforced by database RLS
Communication	Built-in chat	Realtime chat gated by accepted connection
Access control	Application server code	Row-Level Security in PostgreSQL
Hiring workflow	Proposals and contracts	Invitation → developer accepts → client assigns (never automatic)

Criteria	Conventional platforms	CollabriX
Notifications	E-mail and in-app	In-app and real-time, generated by database triggers, per-type preferences
Ranking transparency	Opaque ranking or none	Public formula computed from verified work



Figure 4.1: Splash screen shown while the session is restored

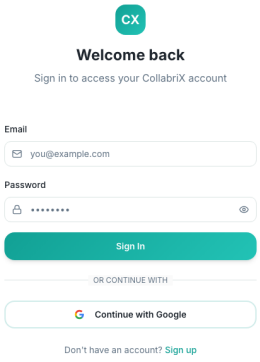


Figure 4.2: Login screen with e-mail/password and Google sign-in

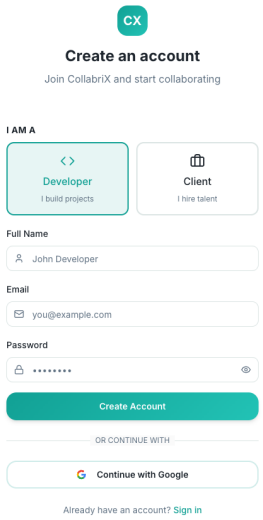
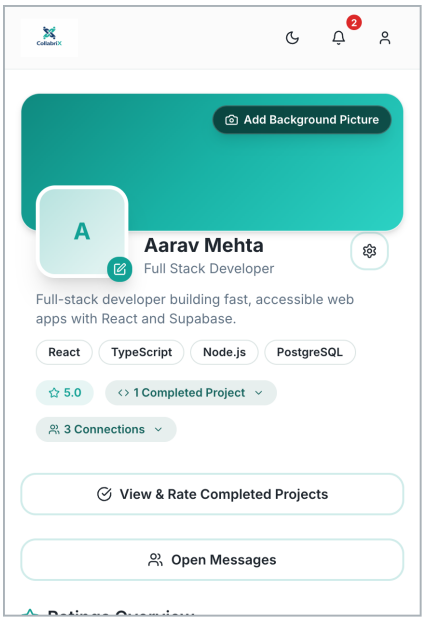
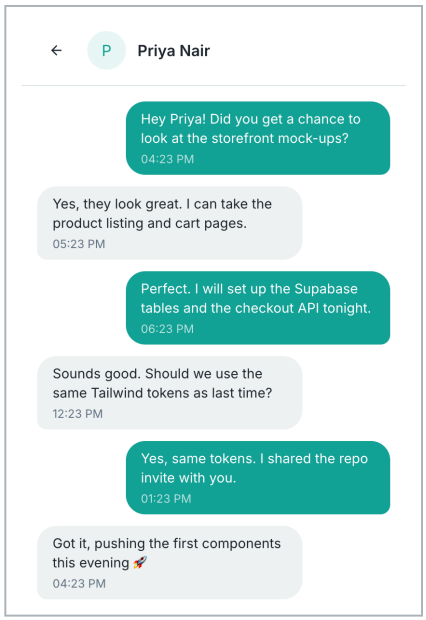


Figure 4.3: Sign-up screen with Developer / Client role selection

The signed-in screens below were captured from the production build at phone width. To avoid writing test data into the live project, the browser's requests to Supabase were answered by a scripted test backend serving a small set of sample users and projects; everything shown is rendered by the real application code.

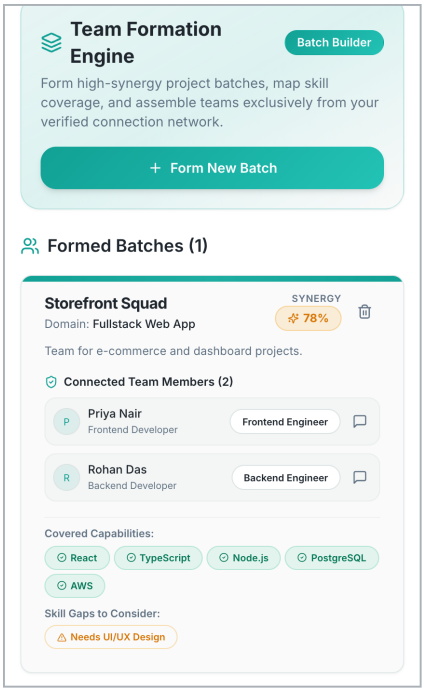


(a) Developer profile

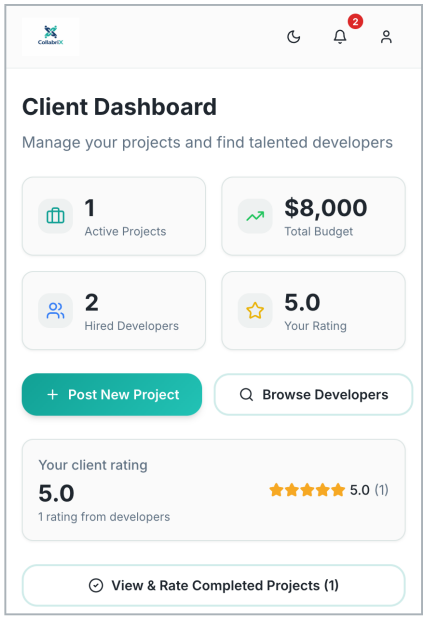


(b) Real-time chat

Figure 4.4: Developer workspace: (a) profile with ratings and badges, (b) real-time chat with a connection

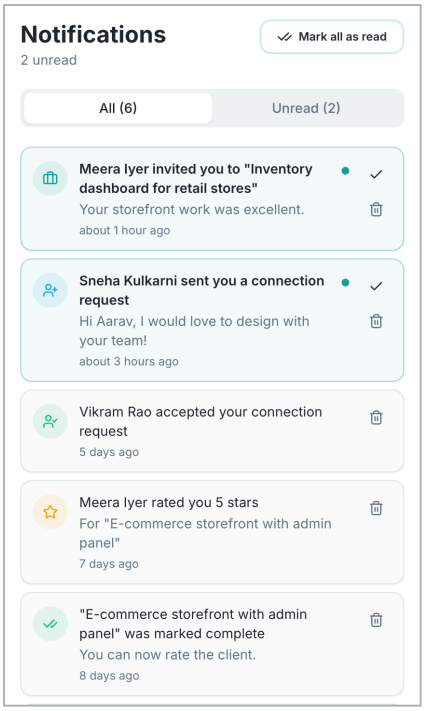


(a) Team Formation Engine

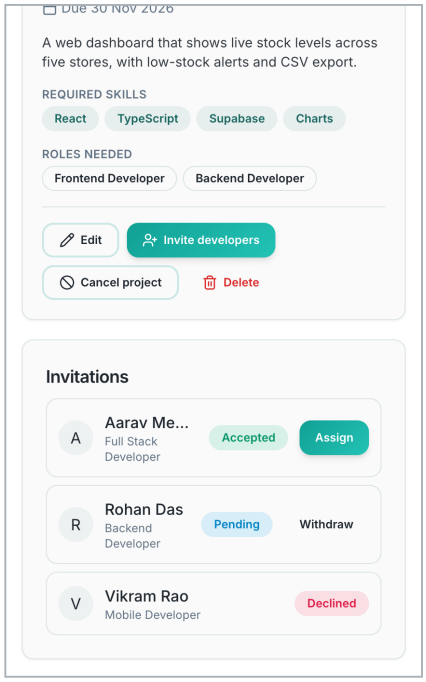


(b) Client dashboard

Figure 4.5: (a) Team Formation Engine with synergy score and skill gaps, (b) client dashboard



(a) Notifications



(b) Project details (client)

Figure 4.6: (a) Notifications page, (b) project details with invitations and the Assign action

4.2.1 Quantitative Results

Testing was carried out at five levels: static analysis, automated user-interface tests, security (RLS) verification with an anonymous client, a database behaviour and security test suite, and performance auditing. All results in this chapter were obtained by actually running the

tools against the project: the original measurements on 6 October 2026 and the re-run after the collaboration extension on 12 October 2026.

A. Static analysis and unit-level checks

The project does not yet include a unit test framework such as Vitest; unit-level correctness was therefore checked through the TypeScript compiler, which type-checks every module against the generated database types, and through ESLint.

Table 4.3: Static analysis results

Check	Command	Result
Type checking	npx tsc --noEmit -p tsconfig.app.json	Passed — 0 errors (before and after)
Linting	npx eslint .	2 warnings, 0 errors (was 49 findings: 14 errors, 35 warnings)
Production build	npx vite build	Succeeded in 1.98 s (was 1.83 s)

On 6 October the lint findings were dominated by missing dependencies in `useEffect` hooks (20 warnings), component files that also export constants (15 warnings) and 10 `no-explicit-any` errors in error-handling code. The rework replaced `any` with typed errors, moved data loading into TanStack Query hooks with explicit dependencies and fixed the remaining rule violations. Two `only-export-components` warnings remain; they affect only hot reloading during development, not the build.

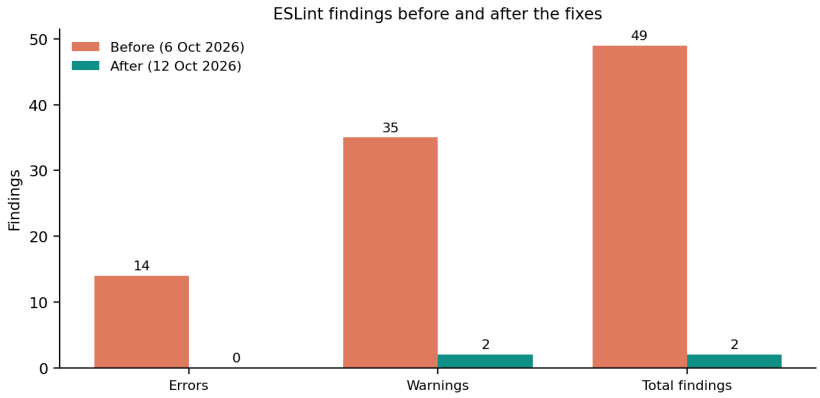


Figure 4.7: ESLint findings before and after the fixes

B. Functional test cases (automated UI tests)

A Puppeteer script drove a headless Chrome browser against the development server to validate the authentication and navigation flows. These tests exercise only client-side logic, so they did not create any accounts on the live backend.

Table 4.4: Automated UI test cases and results

ID	Test case	Expected result	Actual result	Status
TC-01	Open “/” without a session	Redirect to /auth	Redirected to /auth	Pass
TC-02	Log in with malformed e-mail	Submission blocked	Browser validation: “Please include an '@'…”	Pass

ID	Test case	Expected result	Actual result	Status
TC-03	Log in with empty password	Toast “Please enter a password”	Toast shown	Pass
TC-04	Toggle password visibility	Type password → text	password → text	Pass
TC-05	Switch to sign-up mode	Heading and Full Name field	“Create an account”, field shown	Pass
TC-06	Select Client role card	Card highlighted	Highlighted	Pass
TC-07	Sign up with 1-letter name	Toast “Name must be at least 2 characters”	Toast shown	Pass
TC-08	Open an unknown route	404 page	“Oops! Page not found”	Pass
TC-09	Login page at 375 px width	No horizontal overflow	scrollWidth 375 = viewport 375	Pass
TC-10	Open /notifications without a session	Redirect to /auth	Redirected to /auth	Pass
TC-11	Open /settings?tab=account without a session	Redirect to /auth	Redirected to /auth	Pass
TC-12	Open /support without a session	Redirect to /auth	Redirected to /auth	Pass
TC-13	Open /developers/:id without a session	Redirect to /auth	Redirected to /auth	Pass
TC-14	Open /projects/:id without a session	Redirect to /auth	Redirected to /auth	Pass
TC-15	Open a notification deep link (/?tab=connect&view=requests)	Redirect to /auth, no errors	Redirected; no page errors	Pass
TC-16	Google sign-in on login and sign-up	“Continue with Google” on both	Shown on both	Pass
TC-17	Return from Google with an error	Reason shown; URL cleaned	Toast with reason; query removed	Pass
TC-18	Sign-up page at 390 px width	No horizontal overflow	Overflow 0 px	Pass

All eighteen automated test cases passed. TC-01 to TC-09 were run against the development server on 6 October; TC-10 to TC-18, added for the new routes and sign-in behaviour, were run against the production build on 12 October. In TC-02 the request was stopped by the browser's built-in HTML5 e-mail validation before the application's own Zod check, which is the desired outcome.

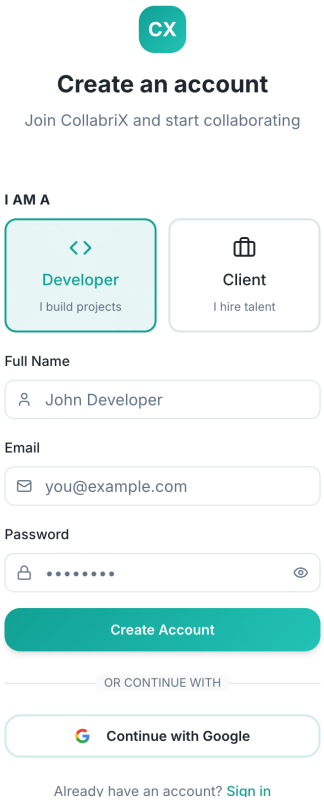


Figure 4.8: Sign-up screen rendered at mobile width (TC-09)

C. Manual test cases for authenticated features

Table 4.5: Manual test cases for authenticated features

ID	Scenario	Expected result
MT-01	Developer A sends a connection request to B	B sees the request with a badge; A sees “Request Sent”
MT-02	B accepts the request	Both see each other under My Connections; Message enabled
MT-03	A sends a message	Message appears in B's open thread without refresh
MT-04	Create a batch with two connections	Synergy score and skill gaps displayed; batch listed
MT-05	Client opens Find Talent with region filter	Only developers from the selected country/ continent listed
MT-06	Participant rates a completed project twice	Second attempt rejected by the unique constraint
MT-07	Client posts a project with a past deadline	Form rejects it with a clear message; nothing saved
MT-08	Client invites developer A; A opens Jobs → Invitations	A sees the invitation and a notification
MT-09	A accepts; client opens the project	Client is notified; A is listed as accepted but not assigned
MT-10	Client assigns A	Status in progress; other pending invitations cancelled; A notified
MT-11	Client marks the project complete; both rate	Both rating buttons appear once; leaderboard score of A increases

ID	Scenario	Expected result
MT-12	Developer disables “Connection requests” notifications	No notification for the next request; the request still appears
MT-13	User opens a support ticket	Ticket number shown; ticket listed with status open
MT-14	User changes the password, then signs out of all devices	Old password rejected; other sessions signed out

These scenarios require two real user accounts on the live backend, so they were not automated in the browser; they serve as the acceptance checklist to be run with two test accounts before the final demonstration. The database rules behind every step are, however, verified automatically by the test suite in Section G.

D. Security testing (Row-Level Security validation)

To validate NFR-1, an anonymous Supabase client (using only the public key that ships in the browser) queried each table and attempted unauthenticated inserts against the live project. Only row counts were recorded.

Table 4.6: RLS validation with an anonymous client

Table / operation	Expected	Observed	Status
SELECT profiles	Readable (public directory)	9 rows visible	Pass
SELECT projects	Readable (public listings)	0 rows (none posted yet)	Pass
SELECT connections	Not readable	0 rows	Pass
SELECT messages	Not readable	0 rows	Pass
SELECT user_roles	Not readable	0 rows	Pass
SELECT ratings tables	Readable but empty	0 rows	Pass
INSERT projects (anon)	Rejected	42501: violates row-level security policy	Pass
INSERT messages (anon)	Rejected	42501: violates row-level security policy	Pass

E. Performance testing

The production build was served locally with `vite preview` and audited with Lighthouse 12 using its default mobile emulation (simulated slow 4G and CPU throttling), on 6 October and again on 12 October 2026 after the extension.

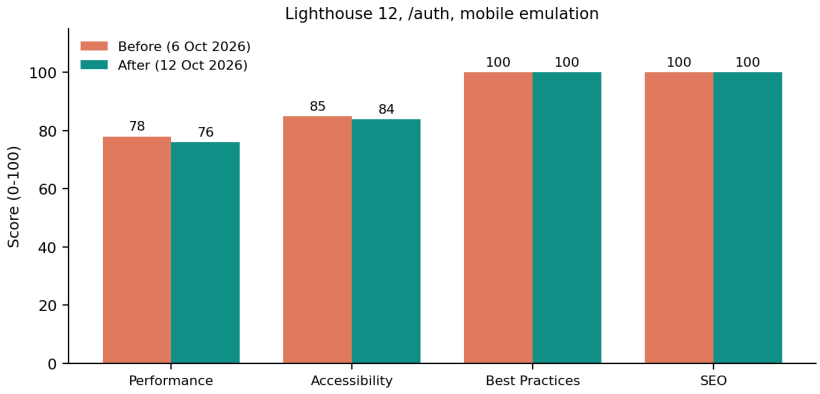


Figure 4.9: Lighthouse category scores before and after the extension

Table 4.7: Lighthouse metrics and bundle sizes

Metric	Value	Remark
First Contentful Paint	3.2 s (was 3.3 s)	Throttled mobile profile
Largest Contentful Paint	4.2 s (unchanged)	Dominated by the JS bundle
Total Blocking Time	30 ms (was 10 ms)	Main thread stays responsive
Cumulative Layout Shift	0	No layout shifts
Speed Index	5.3 s (was 4.0 s)	Varies between runs
JavaScript bundle	931.1 kB (258.1 kB gzip); was 826.8 kB	Single chunk; code-splitting recommended
CSS bundle	78.9 kB (12.9 kB gzip); was 62.5 kB	Tailwind purged output
Logo image	163.8 kB	Candidate for compression / WebP

The main performance bottleneck remains the single JavaScript chunk, which grew from 827 kB to 931 kB with the five new pages and the extended screens; Vite flags it with a size warning. Scores moved by one or two points, which is within the run-to-run variation of a throttled audit. Splitting the client and developer workspaces and the new pages into lazily loaded chunks and compressing the logo are expected to raise the Performance score above 90.

F. Outcome discussion

The security model was compared against an external reference, the OWASP Top 10 [16]: broken access control (A01) is mitigated by RLS together with validation triggers, injection (A03) by parameterised PostgREST queries, insecure design (A04) by enforcing every state transition in the database, and identification failures (A07) by Supabase Auth. Defects found in the first round of testing, such as the Jobs complexity filter that offered *Beginner/ Intermediate/Advanced* while the database stored *Easy/Medium/Hard*, were fixed in the final sprint (Section I).

G. Database behaviour and security test suite

Most of the rules added in the final sprint live in the database, so they were tested there directly. A local PostgreSQL 16 server was prepared with a small stub of the Supabase

environment: the `anon` and `authenticated` roles, an `auth.users` table, and an `auth.uid()` function that reads the user id from the same JWT setting that PostgREST uses. All eleven migrations were then applied in order to an empty database, and a Python script (psycopg) created six users through the real sign-up trigger: four developers and two clients.

Every check executes its SQL statement *as one of these users*, by switching to the `authenticated` role and setting the JWT subject, exactly as Supabase evaluates a request from the browser. A check passes when an allowed action succeeds, when a forbidden insert raises an error, or when a forbidden update or delete affects no rows (RLS filters such statements silently). The live project was never modified.

Table 4.8: Database test suite: areas and results

Area	Checks	Examples of what is verified	Result
Sign-up trigger	1	Profile and role created for every new user	1/1
Connections	21	Self, duplicate and reverse requests refused; only the receiver answers; notifications	21/21
Team batches	10	Only accepted connections; no self or duplicates; members can read, others cannot	10/10
Projects	7	Developers cannot post; no direct completion or assignment; other clients cannot edit	7/7
Invitations	19	Owner-only, developer-only, no duplicates; assignment needs acceptance	19/19
Ratings	19	Only after completion, only the counterpart, once, 1–5, immutable, direction recorded	19/19
Notifications	9	Own rows only; no forging or editing; mark read; internal function not callable	9/9
Settings	4	Own row only; a disabled preference suppresses the notification	4/4
Support tickets	6	Own tickets only; must start open; length checks; status not editable	6/6
Leaderboard	4	Only verified work listed; score formula; not callable anonymously	4/4
Anonymous access	4	No notifications or invitations visible without a session	4/4
Total	**104**		**104/104**

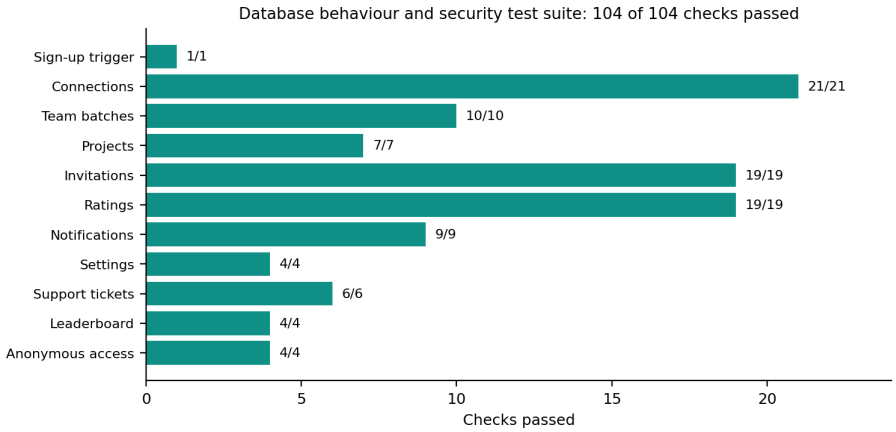


Figure 4.10: Database test suite results by area

H. Migration rehearsal on legacy data

A read-only probe of the live project through its public API showed that the five new tables, the new columns and the new functions did not yet exist, while every earlier migration had been applied. The three pending migrations were therefore combined into one transactional script and rehearsed on a local copy of the live schema seeded with rows that older versions of the application could have written (Table 4.9).

Table 4.9: Upgrade rehearsal with legacy data

Legacy row	Risk	Outcome after the upgrade
Connection from a user to themselves	New CHECK constraint could fail the upgrade	Constraint added as NOT VALID: old row kept, new ones refused
Requests in both directions between the same pair	Duplicate rule	Rule enforced by an insert trigger, so existing rows are untouched
Completed project without completed_at	Leaderboard periods	Counted in “All Time”; excluded from week/month
Ratings in both directions	Direction unknown before the upgrade	Back-filled: 2 client_to_developer, 1 developer_to_client
Rating on a project that was never completed	Score inflation	Kept for history but excluded from the leaderboard
Review longer than 2000 characters	New length CHECK	Constraint added as NOT VALID; new reviews limited
Running the script twice	Partial re-application	Second run fails on the first statement; transaction rolls back; no change

After the upgrade, the leaderboard of the rehearsal database listed exactly one developer with 200 points: 100 for the completed project and 100 for its five-star client rating, while the rating on the unfinished project was correctly ignored. The full test suite also ran on this database; the only failing check was the one that expects the leaderboard to contain a single developer, because the seeded legacy developer was a second, legitimate entry.

I. Defects fixed in the final sprint

Table 4.10 lists the most important defects found by testing and code review of the 6 October version, their root cause and how each was resolved.

Table 4.10: Defects, root causes and fixes

Defect	Root cause	Fix	Verified by
Clients could not post projects	No form or insert flow in the client UI	Validated Post Project dialog; insert policy for clients	MT-07, DB suite
No way to assign or complete projects	Lifecycle existed only in the schema	Invitations, assign_developer, complete_project	DB suite, MT-08–MT-11
Unread message counts never decreased	Only the sender could update messages	mark_messages_read function	Manual check
Team batches only in one browser	Stored in localStorage	team_batches table with trigger validation	DB suite
Demo connections and a default 4.8★ rating	Mock data shown when lists were empty	Removed; honest empty states	Code review
Leaderboard ranked unrated users at 4.5	Baseline average in client code	Server-side get_leaderboard on verified work	DB suite
Jobs complexity filter returned nothing	UI values differed from database values	Shared Easy/Medium/Hard values	Manual check
Ratings could be mixed up by direction	No direction stored; wrong “already rated” check	rating_type column and per-direction checks	DB suite
Fake success messages	Toasts shown before or regardless of results	Await results; friendlyError	Code review
Duplicate Messages entry for clients	Quick action duplicated the tab	Removed the duplicate	Manual check
Settings, Support and bell did nothing	Buttons without handlers	Real pages and routes	TC-10–TC-12
Sign-in tokens logged to the console	Debug logging	Removed	Code review
Google sign-in failures were silent	Redirect error dropped by routing	Reason captured and shown	TC-17
Name and avatar hidden on profile headers	Positioned cover image painted over the profile row	Profile row raised above the cover	Screenshot (Figure 4.4)
Unreadable “Batch Builder” badge	Teal text on a teal background	Contrasting text colour, no wrapping	Screenshot (Figure 4.5)

Chapter 5

Conclusion and Future Works

5.1 Conclusion

CollabriX demonstrates that a freelance platform can be designed around teams rather than individuals. The prototype delivers two complete role-based workspaces, guided onboarding, developer discovery with faceted filters, a request-and-accept connection workflow, real-time messaging, a team formation engine with an explainable synergy score, and, after the final sprint, a complete project lifecycle from posting through invitation, assignment and completion to two-way ratings, with real-time notifications, settings and support. The entire authorisation model lives in PostgreSQL: RLS decides who may touch a row, triggers decide which changes are valid, and security-definer functions perform the multi-step operations. 104 automated database checks confirm that these rules hold.

From an engineering perspective, the project gave the team practical experience with a modern TypeScript front-end stack, relational schema design, SQL-based security, realtime systems, agile delivery in short sprints and systematic testing.

Table 5.1: Project goals versus accomplishments

Objective	Accomplishment	Status
Role-based responsive SPA	Developer (5 tabs) and client (4 tabs) workspaces; works at 375 px	Achieved
Peer discovery and connections	Search, type/skill/region filters, request–accept workflow	Achieved
Real-time messaging	WebSocket subscriptions per connection; shared inbox	Achieved
Team batches with synergy score	Built from accepted connections with roles and gaps; stored in the database with trigger validation; members notified	Achieved
Completion-gated reputation	RLS-enforced ratings, leaderboard and badges; rating UI on completed projects	Achieved
Database-level security	RLS on all 12 tables and storage; triggers and functions for every transition; 104 automated checks	Achieved
Evaluation and documentation	UI (18 cases), RLS, database, static and performance tests; limitations documented	Achieved
Complete project lifecycle	Post, invite, assign after acceptance, complete, rate in both directions	Achieved
Real-time notifications	10 event types, per-type preferences, read state, deep links	Achieved
Settings and support	Profile, notifications, privacy, account security; ticketed support	Achieved

5.1.1 Lessons Learned

- **Put the rules in the database first.** Writing RLS policies before the screens forced us to decide early who may read and write each table, and it made the front end simpler because it never has to re-check permissions.
- **Realtime needs clean-up.** Every Supabase channel opened in a React effect must be removed when the component unmounts; otherwise duplicate subscriptions and repeated notifications appear.
- **Demo data hides gaps.** Sample connections and default ratings made early demos look complete, but they also concealed missing workflows. We now prefer honest empty states.
- **Keep the data model aligned with the UI.** The complexity-filter mismatch showed that enumerated values should be defined once and shared between SQL and TypeScript.
- **Generated code still needs review.** AI-assisted scaffolding (Lovable) sped up UI work, but generated files need the same review as hand-written code; for example, one onboarding modal was rendered twice on the client profile screen.
- **Make state transitions explicit.** Drawing the state diagrams for connections, projects and invitations exposed several transitions the first version allowed by accident, such as reopening a completed project.
- **Never show success you have not confirmed.** Waiting for the database's answer is slightly slower to implement but removes a whole class of confusing bugs.
- **Test the database like code.** Running the migrations against a disposable PostgreSQL instance made it possible to test security rules as different users without risking live data.

5.2 Limitations

Table 5.2: Known limitations of the current prototype

Area	Limitation
Deployment	New migrations must be applied by an administrator in the Supabase SQL editor; a combined, transactional script is provided.
Live verification	Realtime delivery and the two-account flows (MT-01–MT-14) are verified at database level but not yet end to end in the browser on the live project.
Google sign-in	Creation of new accounts through Google is still being diagnosed; the failure reason is now shown to the user.
Onboarding	LinkedIn and résumé options do not extract data; they open the manual form.
Support	There is no staff console; ticket status is changed by administrators in SQL.
Leaderboard	The score rewards volume of verified work, so many modest ratings can outrank a few excellent ones.
Code quality	No unit-test framework for React components yet (the database suite covers the rules); 2 lint warnings.
Performance	Single 931 kB JavaScript bundle; Lighthouse Performance 76.

5.3 Future Prospectus

1. **Team proposals:** let a team batch apply to a project as one unit, with roles and a revenue split per member.
2. **Staff console:** role-protected screens for support tickets, user reports and moderation.
3. **End-to-end tests:** Playwright scenarios with two accounts against a disposable Supabase project in continuous integration, covering MT-01 to MT-14.
4. **Smarter team suggestions:** recommend connections that fill skill gaps, weighting ratings and current workload, following [5] and [11].
5. **Multi-dimensional, double-blind ratings:** separate scores for quality, communication and timeliness, revealed only after both parties submit, to counter reputation inflation [8], and a Bayesian average so that a few ratings do not dominate the leaderboard.
6. **Payments and escrow:** milestone-based payments through a payment gateway such as Razorpay or Stripe.
7. **Quality and performance:** code-split the workspaces and pages with React.lazy, compress the logo to WebP, and add Vitest unit tests for the hooks.

5.3.1 Possible Integration with Other Tools

- **GitHub** — link repositories to profiles and show contribution activity as verifiable evidence of skills.
- **Google Calendar / Meet** — schedule and join client meetings directly from a conversation.
- **Slack or Discord webhooks** — mirror team notifications for users who prefer those tools.
- **LinkedIn** — official OAuth-based profile import in place of the current link field.
- **E-mail and push delivery** — send the same notification rows by e-mail or web push through Supabase Edge Functions, respecting the existing preferences.

Bibliography

[1] J. J. Horton, "Online labor markets," in *Proc. 6th Int. Workshop on Internet and Network Economics (WINE)*, LNCS, vol. 6484, Springer, 2010, pp. 515–522.

[2] A. Kittur, J. V. Nickerson, M. Bernstein, E. Gerber, A. Shaw, J. Zimmerman, M. Lease, and J. Horton, "The future of crowd work," in *Proc. ACM Conf. Computer Supported Cooperative Work (CSCW)*, 2013, pp. 1301–1318.

[3] P. Resnick, K. Kuwabara, R. Zeckhauser, and E. Friedman, "Reputation systems," *Communications of the ACM*, vol. 43, no. 12, pp. 45–48, 2000.

[4] A. Jøsang, R. Ismail, and C. Boyd, "A survey of trust and reputation systems for online service provision," *Decision Support Systems*, vol. 43, no. 2, pp. 618–644, 2007.

[5] T. Lappas, K. Liu, and E. Terzi, "Finding a team of experts in social networks," in *Proc. 15th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2009, pp. 467–476.

[6] D. Retelny, S. Robaszkiewicz, A. To, W. S. Lasecki, J. Patel, N. Rahmati, T. Doshi, M. Valentine, and M. S. Bernstein, "Expert crowdsourcing with flash teams," in *Proc. 27th ACM Symp. User Interface Software and Technology (UIST)*, 2014, pp. 75–85.

[7] M. A. Valentine, D. Retelny, A. To, N. Rahmati, T. Doshi, and M. S. Bernstein, "Flash organizations: Crowdsourcing complex work by structuring crowds as organizations," in *Proc. CHI Conf. Human Factors in Computing Systems*, 2017, pp. 3523–3537.

[8] A. Filippas, J. J. Horton, and J. Golden, "Reputation inflation," *Marketing Science*, vol. 41, no. 4, pp. 733–745, 2022.

[9] M. Kokkodis and P. G. Ipeirotis, "Reputation transferability in online labor markets," *Management Science*, vol. 62, no. 6, pp. 1687–1706, 2016.

[10] A. J. Wood, M. Graham, V. Lehdonvirta, and I. Hjorth, "Good gig, bad gig: Autonomy and algorithmic control in the global gig economy," *Work, Employment and Society*, vol. 33, no. 1, pp. 56–75, 2019.

[11] A. Anagnostopoulos, L. Becchetti, C. Castillo, A. Gionis, and S. Leonardi, "Online team formation in social networks," in *Proc. 21st Int. Conf. World Wide Web (WWW)*, 2012, pp. 839–848.

[12] C. Dellarocas, "The digitization of word of mouth: Promise and challenges of online feedback mechanisms," *Management Science*, vol. 49, no. 10, pp. 1407–1424, 2003.

[13] I. Fette and A. Melnikov, "The WebSocket Protocol," RFC 6455, IETF, Dec. 2011.

[14] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, May 2015.

[15] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Univ. of California, Irvine, 2000.

[16] OWASP Foundation, "OWASP Top 10:2021." [Online]. Available: https://owasp.org/Top10/

[17] Supabase, "Row Level Security," Supabase Documentation. [Online]. Available: https://supabase.com/docs/guides/database/postgres/row-level-security

[18] Meta Open Source, "React — The library for web and native user interfaces." [Online]. Available: https://react.dev

[19] Vite, "Vite — Next generation frontend tooling." [Online]. Available: https://vitejs.dev

[20] TanStack, "TanStack Query documentation." [Online]. Available: https://tanstack.com/query

[21] Tailwind Labs, "Tailwind CSS documentation." [Online]. Available: https://tailwindcss.com/docs

[22] The PostgreSQL Global Development Group, "Row Security Policies," PostgreSQL Documentation. [Online]. Available: https://www.postgresql.org/docs/current/ddl-rowsecurity.html

[23] Google, "Lighthouse overview," Chrome for Developers. [Online]. Available: https://developer.chrome.com/docs/lighthouse/overview

[24] K. Beck *et al.*, "Manifesto for Agile Software Development," 2001. [Online]. Available: https://agilemanifesto.org

[25] IEEE Recommended Practice for Software Requirements Specifications, IEEE Std 830-1998, 1998.

[26] D. Hardt, "The OAuth 2.0 Authorization Framework," RFC 6749, IETF, Oct. 2012.